

Vers le Déploiement Continu

R5.07 - Automatisation de la chaîne de production.

Nicolas Ferry

Inspiré des cours de S. Lavirotte & G. Urvoy-Keller

& <https://blog.stephane-robert.info/docs/infra-as-code/gestion-de-configuration/ansible/>

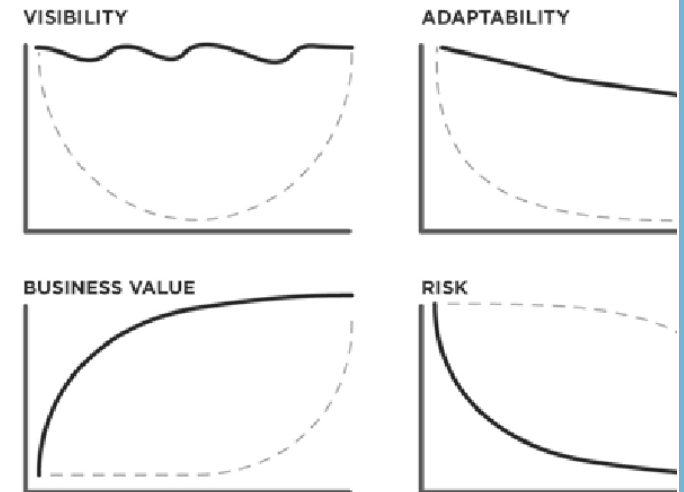
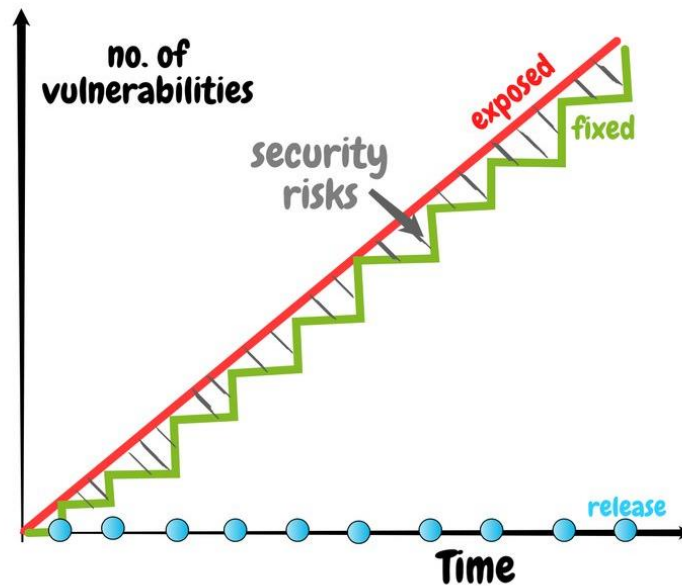
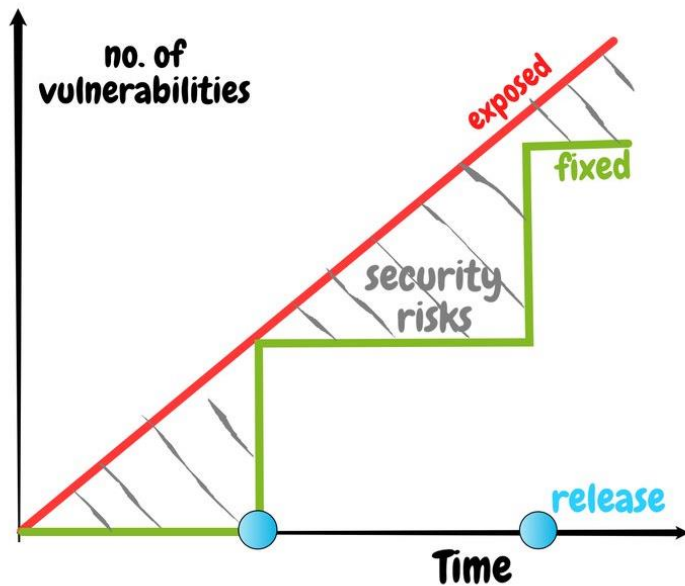
L'objectif de l'intégration continue

- **Livrer** rapidement aux clients les changements apportés à une solution logicielle.
- Tout en garantissant / **maintenant** / **améliorant** sa qualité.



Mais aussi

- Infrastructure, requirements, context, might **change frequently**
- The ability of systems to **continuously evolve** to their environment /... is decisive to ensure and increase their **trustworthiness and quality**.
 - Efficient only if **small, iterative and incremental evolution steps**.

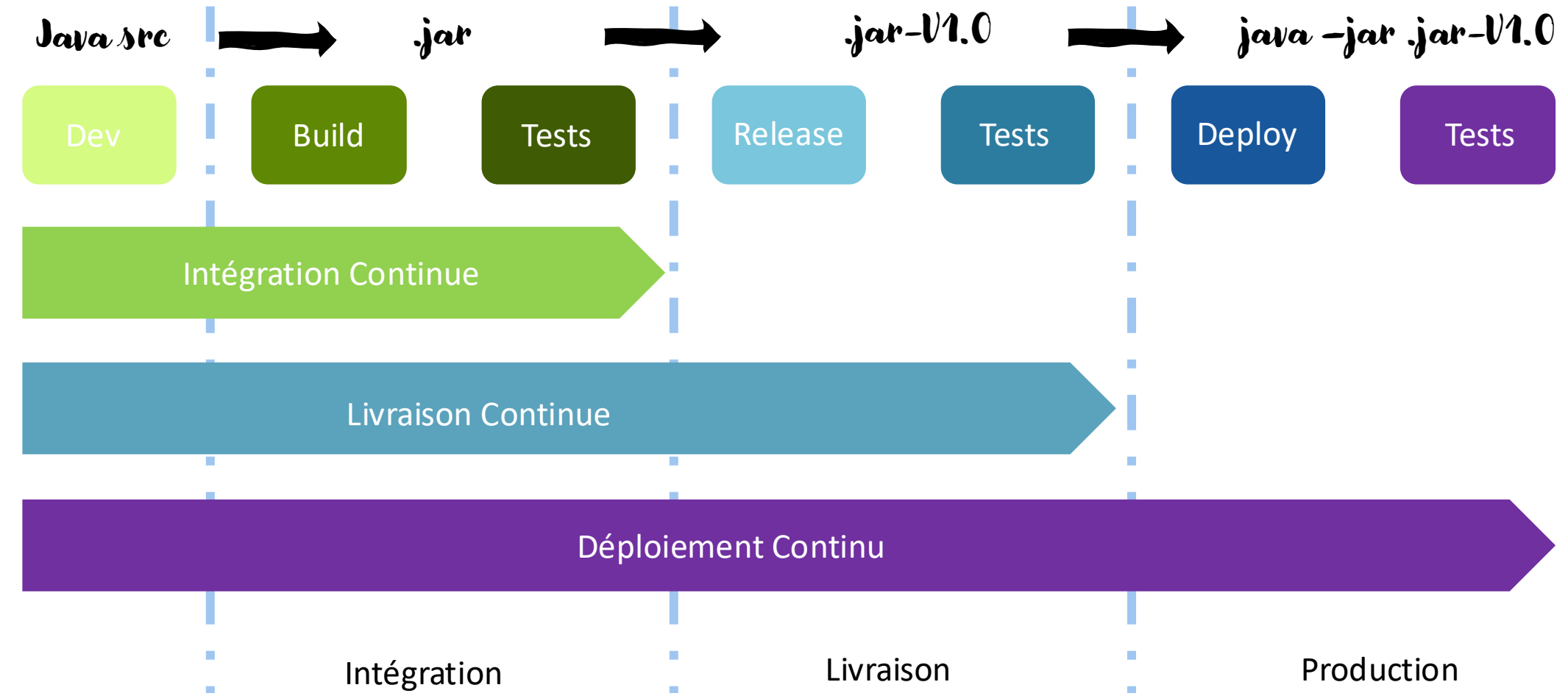


<https://doi.org/10.1145/1980022.1980188>

Livrer (voir déployer) et maintenir

- Pour livrer et mettre à jour rapidement du logiciel, il faut pouvoir faire de l'itératif et incrémental, et donc :
 - Avoir une architecture / conception pour.
 - Une méthodologie pour.
- ...
- Réduire les interventions manuelles et **automatiser** livraison et maintenance.
- **Observer, analyser** le comportement de son logiciel.

Automatiser



Observer et analyser

- La condition *sine qua non* pour pouvoir automatiser le processus de livraison / déploiement : **Confiance**
 1. Avoir confiance en son processus / mécanisme de livraison.
 2. Que les clients aient confiance dans les livraisons (i.e., si une fois sur deux ce que je livre ne marche pas ...)
- Pour construire la confiance:
 - Tests
 - Répétabilité du processus
 - Analyse du risque
 - Traçabilité
 - ...

Intégration continue

« Une méthode de développement logiciel dans laquelle le logiciel est reconstruit et testé à chaque modification apportée par un programmeur. »

Pourquoi c'est utile : un exemple

- **Dans une heure un groupe d'investisseur passe au bureau pour voir la version la plus récente de votre système.**
- *Sans intégration continue, un scénario probable:*
 - Panique à bord: l'état du prototype n'est, pas clair, tous les éléments ont-ils été testés ensemble ? Intégration de dernière minute bancale : la démo de marche pas
- *Idéalement:*
 - Pas de soucis, une version avec les dernière nouveautés est toujours packagé et testée.

Comment mettre en place l'IC

1. Un dépôt de code source
2. Un mécanisme pour construire automatiquement du logiciel
3. Une « plateforme » pour exécuter des tests
4. Un outil qui fait la glue entre **1,2, et 3.**

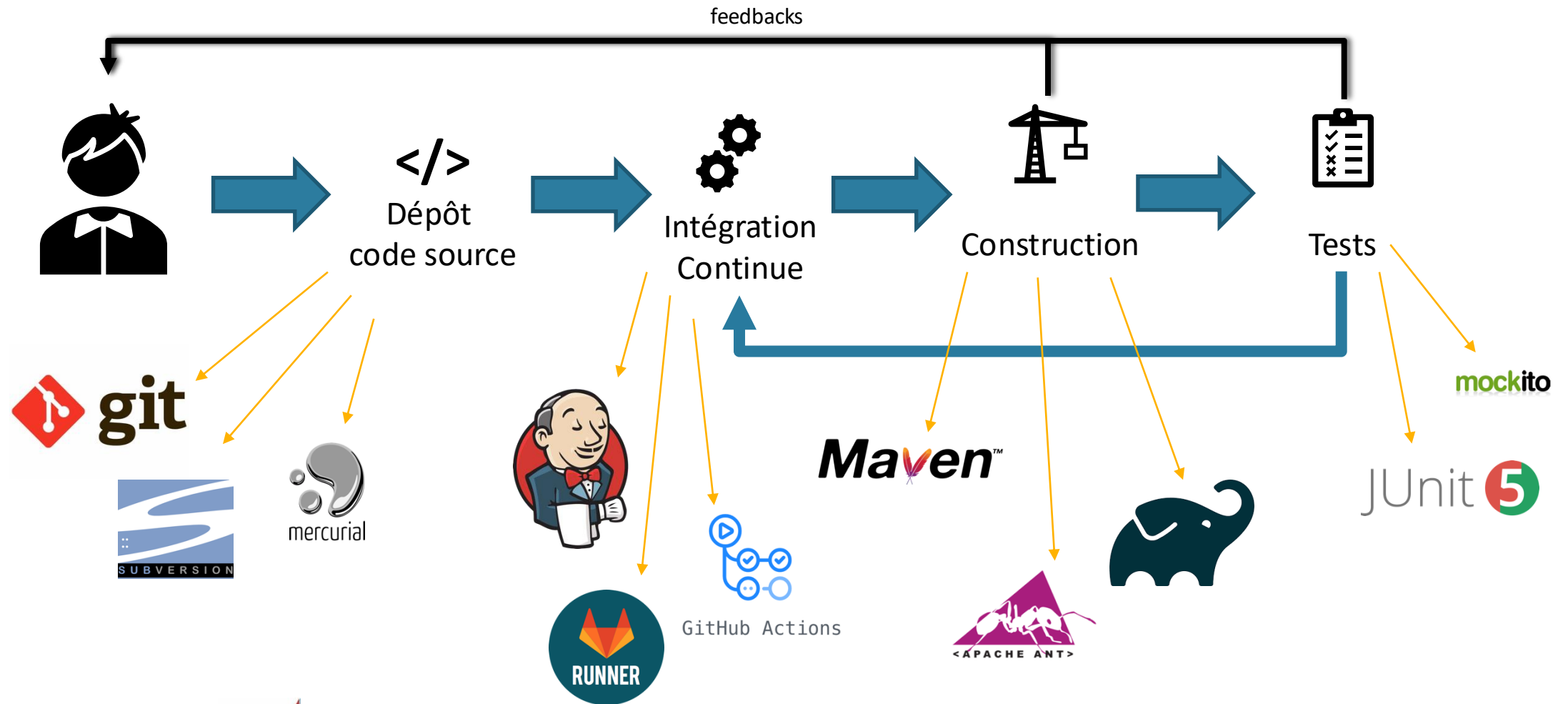
ET

- Une méthodologie/procédure commune pour travailler et pousser ses modifications.
- Une architecture et une design permettant de faire de l'incrémentale.

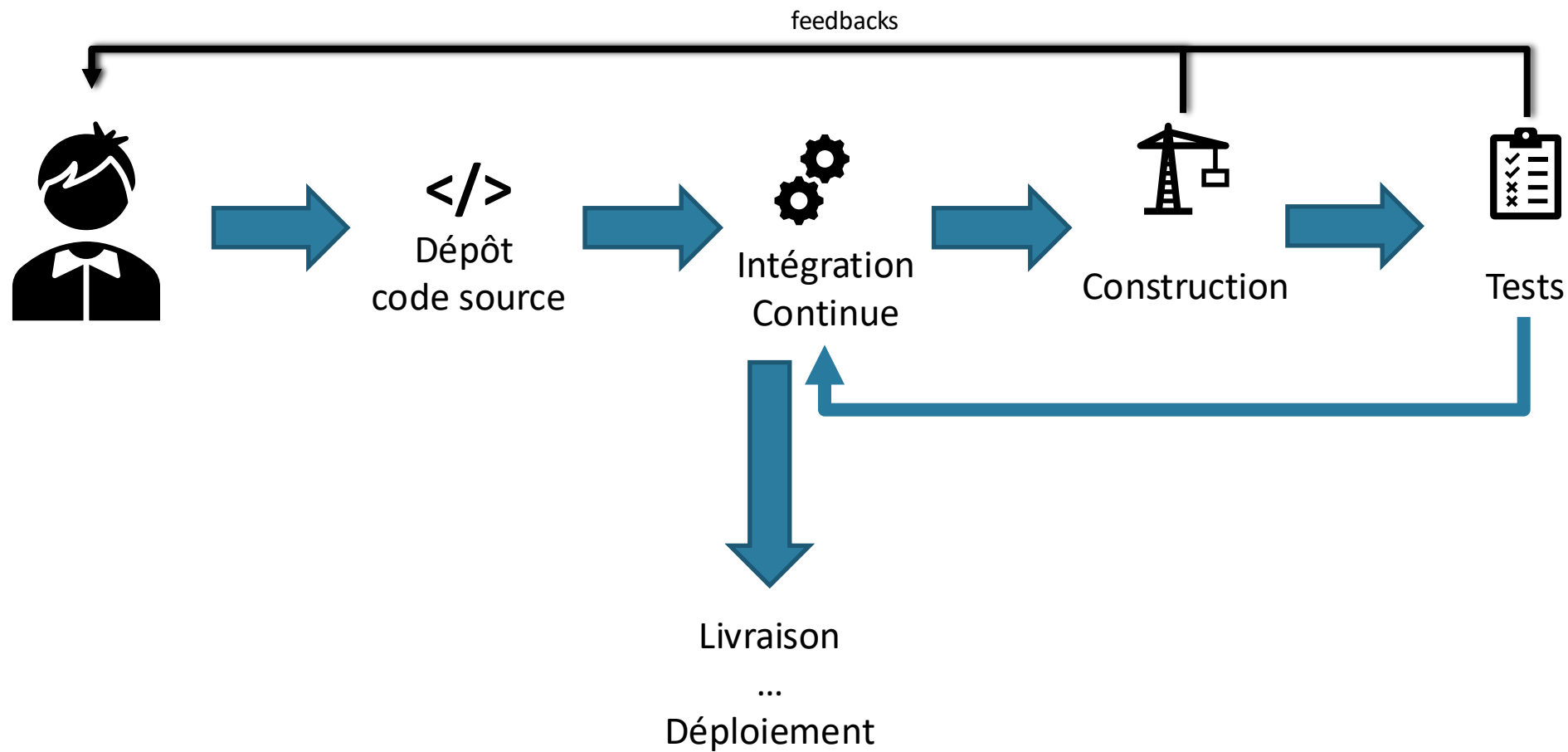
Un outil qui fait la glue entre 1,2, et 3.

- Des outils tels que Jenkins, GitHub action, Gitlab runner.
 - On parle souvent de *CI server* ou de *server d'intégration*.
- Ces outils permettent de créer des workflows / pipeline de tâches à automatiser.
- Permet de faire le lien entre les activités que l'on a déjà vu!
- E.g.:
 - Quand on fait un merge sur la branche main...
 - ... on lance la construction de notre appli ...
 - ... en mode release ...
 - ... et on la déploie ...

Pipeline et outils



Pipeline et outils



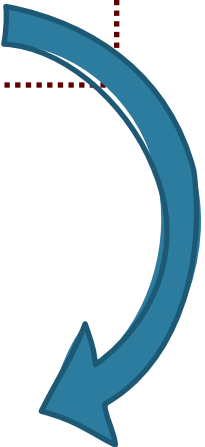
Livrer (voir déployer) et maintenir

- Pour livrer et mettre à jour rapidement du logiciel, il faut pouvoir faire de l'itératif et incrémental, et donc :
 - Avoir une architecture / conception pour.
 - Une méthodologie pour.

...

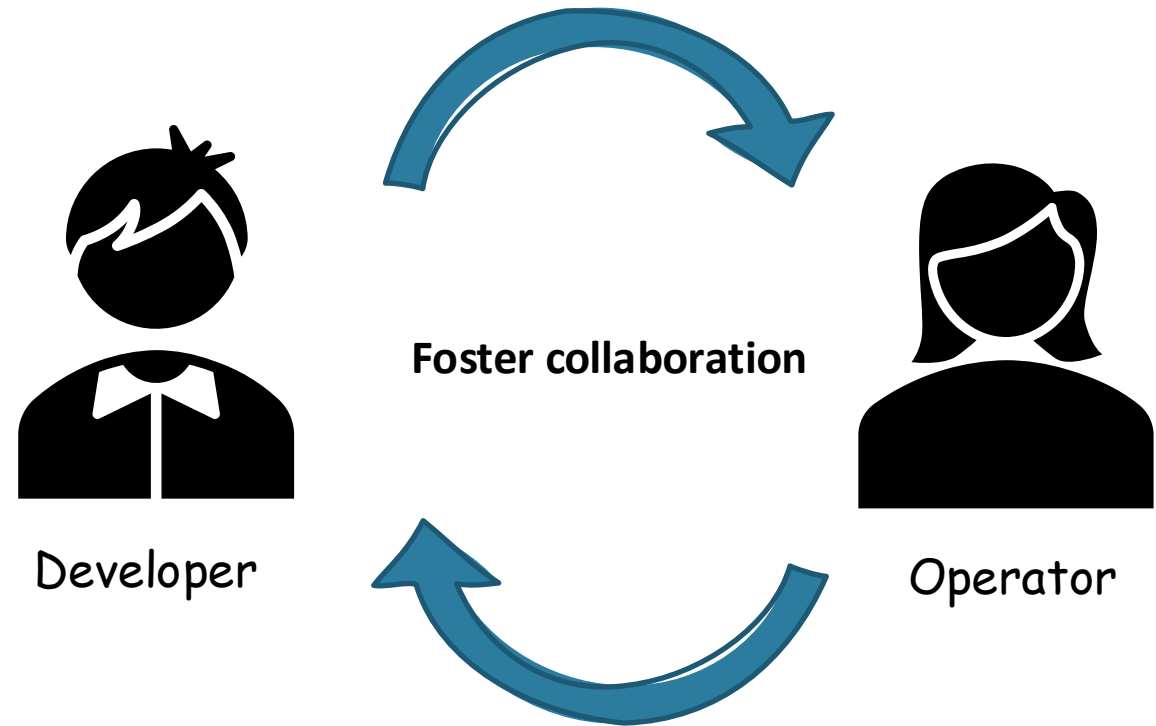
- Réduire les interventions manuelles et **automatiser** livraison et maintenance.
- **Observer, analyser** le comportement de son logiciel.

Developpeurs?
Admin?
Les deux?

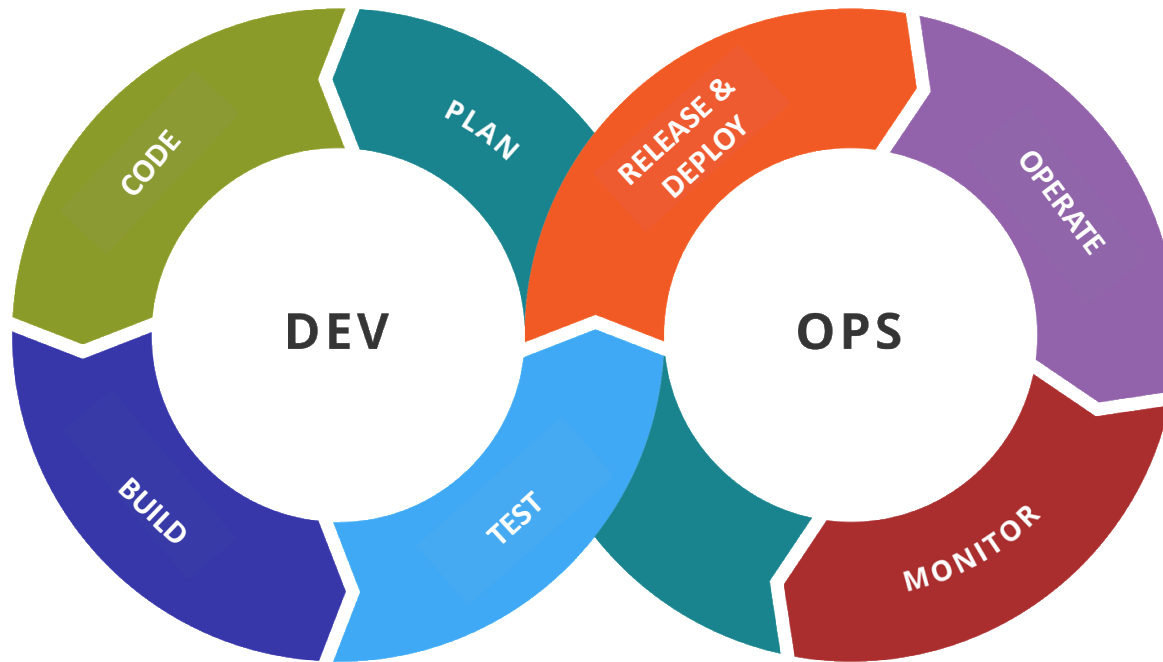


DevOps

- **Dev** for development team
- **Ops** for operation team
- *Sometimes:*
- **Dev** for development
- **Ops** for operation



DevOps in a nutshell



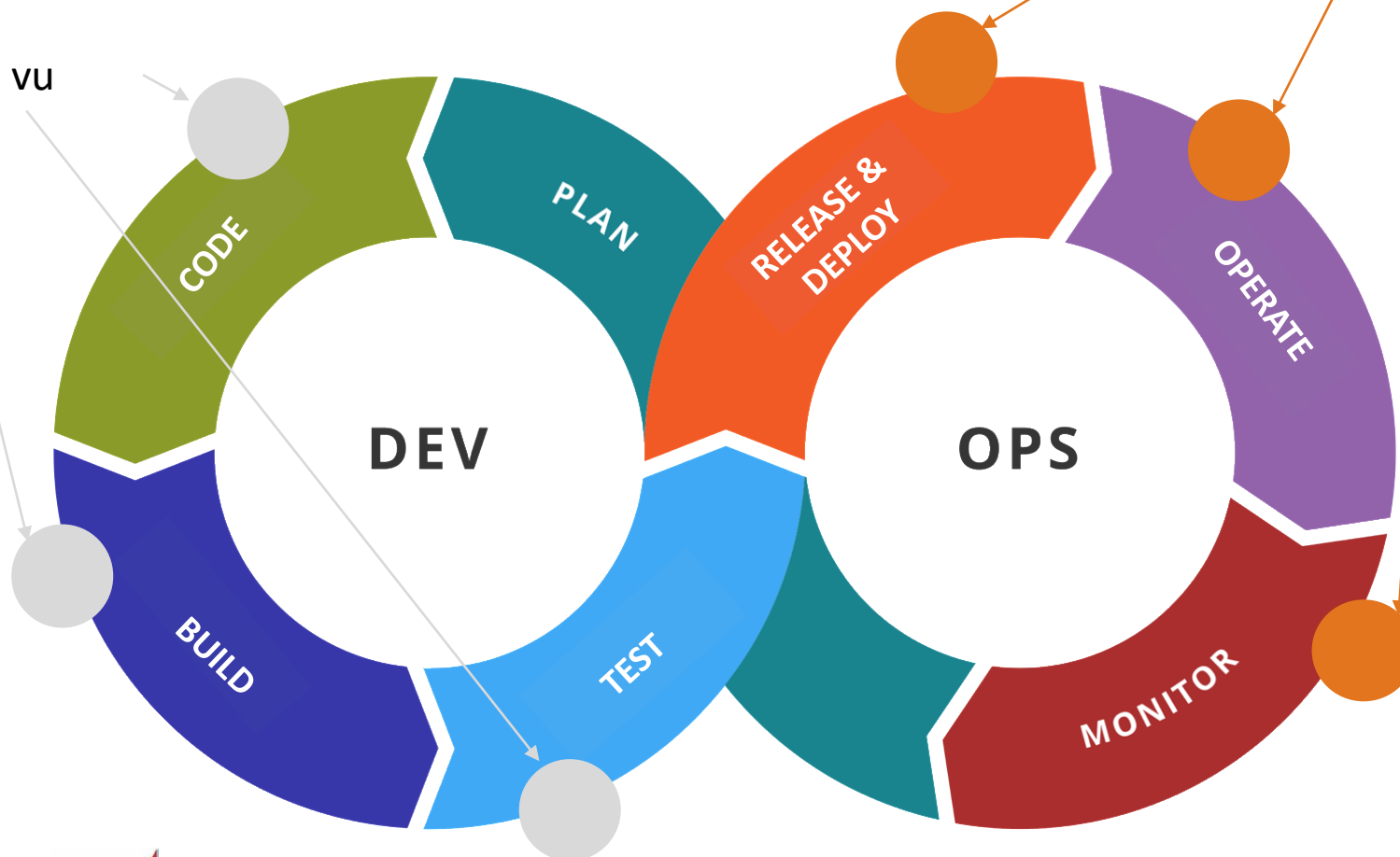
- **DevOps core values:**

- **Culture:** Practice of operation and development engineers participating together in the entire system lifecycle
- **Automation:** Infrastructure as Code, Continuous Delivery
- **Feedback:** Measure everything, feedback from Ops to Dev

Cette année

Ce que vous avez déjà vu

Ce que l'on va approfondir
cette année



Objectif du cours

- **Ce cours fait directement suite à R2.03 et R4.02.**
 - Dans lesquels nous avons vu comment mettre en place un pipeline de livraison continue.
- Cette fois nous allons nous concentrer sur le **déploiement continu** et le **monitoring d'application**.
- A la fin de ce cours vous saurez faire :
 1. Des pipelines de déploiement et de la mise à jour continue.
 2. Mettre en place des piles de monitoring pour observer la santé de vos applications.
 3. Passer vos applications à l'échelle.

Contenu

- 1. Déploiement continu**
(avec Docker et Docker-compose)
- 2. Configuration Management**
(avec Ansible)
- 3. Techniques de mise à jour et de passage à l'échelle**
(avec Docker Swarm)
- 4. Monitoring d'application**
(avec Prometheus)

Evaluation

- **Via un projet :**
 - Rendu d'un dépôt (coeff. 1).
 - Rendu d'une documentation.
 - Une soutenance (coeff. 1).
- **Un contrôle sur papier (60 minutes – coeff. 1) :**
 - Sur ce que vous avez compris.
 - Et un peu de code (mais on reste flexible vis à vis de la syntaxe).

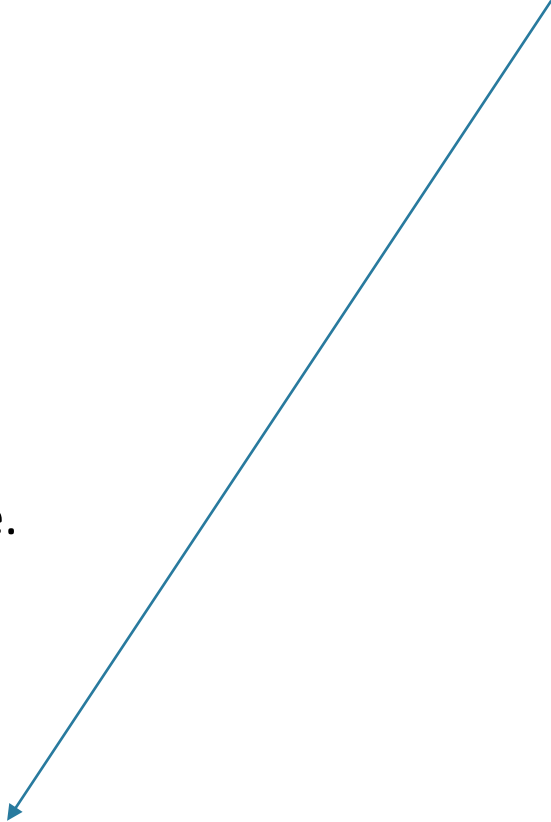
Le projet - Contexte

- Projet commun aux ressources suivantes:
 - R5.01 – Mngt. d'une équipe de projet info.
 - R5.13 – Economie durable et numérique.
 - R5.04 – Programmation avancée.
 - R5.07 – Auto. chaine de production.

} Mme. Rochard
} M Courtoisier
} Mme. Peraldi
} Avec Moi

Le projet - Contexte

Attention tous les membres d'un groupe doivent contribuer au pipeline !



- **En R5.01 & R5.13 :**
 - Vous constituerez les groupes.
 - Organiserez votre travail au sein du groupe.
 - Prendrez en compte le côté « business ».
- **Du point de vue de R5.04 :**
 - Vous utiliserez ce que nous avons vu en cours. Votre code sera évalué.
- **Du point de vue de R5.07 :**
 - 2 séances de TP dédiées au projet.
 - Mise en place de la chaine de production, détails à suivre.

Projet - Sujet

« Le projet BUT3-IoT porte sur l'exploration des adaptations possibles de l'être humain à son environnement grâce à l'IoT. »

Cf. Lettre de commande

Le projet - Technologies

- Tant que vous **respectez les contraintes/consignes** vous êtes **libre de choisir les technologies** de votre choix.
- **Attention :**
 - Cela veut aussi dire que vous devez être un **maximum autonome** sur les problèmes liés à vos choix technos (i.e., je ne connais pas toutes les technos!).
 - Les choix des technos doivent être un minimum **motivés** et une **architecture adaptée** doit être proposée.
 - Exemple: Une base de données VS un fichier JSON.

Le projet – Consignes générales

- **Composition** : en groupes (taille et composition à définir en R5.01)
- **Démarre** : aujourd'hui
- **Date de rendu** : 12/12
- **Date de soutenance** : 13/01 après-midi
- **Contenu du rendu (détaillé ci-après)** :
 - Votre dépôt GitLab.
 - Votre documentation.

Consignes pour R5.07

- **Mise en place d'un pipeline de déploiement continu avec GitLab qui doit:**
 1. Lancer automatiquement des tests unitaires.
 2. Construire l'application.
 3. Publier vos résultats vers un Sonarqube.
 4. Publier une/des image(s) Docker de votre application sur Dockerhub.
 5. L'application doit pouvoir être lancée avec un fichier Docker-compose.
 6. L'application doit être déployée automatiquement sur Azure/ou autre. Possibilité d'utiliser Ansible pour configurer l'hôte.
 7. BONUS : Les services qui le nécessitent doivent pouvoir passer à l'échelle.
 8. Mise en place d'une pile de monitoring.
- Vous définissez votre stratégie d'utilisation de Git.
- Vous définissez quand est déclenché votre pipeline.
- Accompagné d'une documentation.

Elements d'évaluation pour R5.07

- **Le bon fonctionnement du pipeline.**
- **Qualité de la documentation, explication de vos choix de conception.**
- **GIT (GitLab IUT):**
 - Stratégie d'utilisation du dépôt Git.
 - L'utilisation de Git (chaque étudiant est évalué individuellement). Cela inclus : Commit réguliers et bien documentés; Le respect de la stratégie d'utilisation de Git.
- **Tests :**
 - Qualité des tests.
 - Utilisation de mocks, autres types de tests
- **Construction :**
 - Bonne gestion des dépendances et modularité de la conception
 - Qualité des images Docker construites (taille, pas d'ip en dur, etc.)
 - Le statut des rapports SonarQube
- **Déploiement :**
 - Qualité du Docker-compose (pas d'ip en dur, réseaux, etc.) / fichiers Ansible.
 - Explications du mécanisme de passage à l'échelle si mis en place.
 - Explications du mécanisme de Monitoring.

Retour très rapide sur la livraison automatique

Rappel GitLab-Runner

- Le serveur d'intégration de GitLab.
 - Permet l'automatisation des :
 1. Tests,
 2. Construction & packaging,
 3. Livraison
 4. Déploiement, etc.
 - Ces tâches sont exécutées dans des runners ...
 - ... qui sont eux-mêmes exécutés dans des containers.
- Les tâches et l'ordre dans lesquelles elles doivent être exécutées sont spécifiés dans un manifeste.
 - Un fichier au format YAML.
 - Nommé `.gitlab-ci.yml`

Manifeste : les concepts

```
default:
  image: ruby:3.0

stages:
  - build
  - test

build-code-job:
  stage: build
  script:
    - echo "Check the ruby version, then build some Ruby project files:"
    - ruby -v
    - rake

test-code-job1:
  stage: test
  script:
    - echo "If the files are built successfully, test some files with one command:"
    - rake test1

test-code-job2:
  stage: test
  script:
    - echo "If the files are built successfully, test other files with a different command:"
    - rake test2
```

La partie « default » permet de définir les valeurs par défauts qui seront utilisées dans les « jobs », c'est-à-dire les scripts qui seront exécutés. Entre autre on définit dans quel container nos scripts doivent être exécutés.

Les stages sont les grandes étapes du pipeline. A chaque stage peut être associés plusieurs jobs. Dans un stage, par défaut les jobs sont exécutés en // Les stages sont réalisés dans l'ordre de leur déclaration. On passe au stage suivant que si le stage courant réussit

Déclaration d'un job (un script)

Associé au stage test

Le script à exécuter.

Manifeste : les concepts

- Un **job** doit a minima avoir un **script** ...
- ... et peut en contenir plusieurs.
- Peut avoir n'importe quel nom sauf mot clés réservés.
- Peut être déclenché sur l'occurrence de certains événements.

```
job:1:  
  script: echo 'my first job'  
  
job:2:  
  script: echo 'my second job'
```

```
job:commands:  
  script: # Exécution de deux commandes  
    - printenv  
    - echo $USER
```

```
stages:  
  - build  
  - test  
  - report  
  - clean  
  
job:build:  
  stage: build  
  script:  
    - make build  
  
job:test:  
  stage: test  
  script:  
    - make test  
  when: on_success # s'exécutera uniquement si le job `job:build` passe  
  
job:report:  
  stage: report  
  script:  
    - make_report  
  when: on_failure # s'exécutera si le job `job:build` ou `job:test` ne passe pas  
  
job:clean:  
  stage: clean  
  script:  
    - make clean # s'exécutera quoi qu'il se passe  
  when: always
```

Manifeste : les concepts

- Les jobs peuvent produire un fichier ou une archive de fichiers/répertoires. On parle de `job artifact`. Il est possible de mettre ces `job artifacts` à disposition (livraison !)

```
pdf:  
  script: xelatex mycv.tex  
  artifacts:  
    paths:  
      - mycv.pdf  
  expire_in: 1 week
```

- Il est possible de construire des variables utilisables dans tout le manifeste

```
variables: # Déclaration de variables pour tous les `job`  
  SYMFONY_ENV: prod  
  
build:  
  script: echo ${SYMFONY_ENV} # Affichera "prod"
```

Rappel GitLab-Runner

```
default:  
  image: maven:3.8.7-openjdk-18  
  cache:  
    key: "CACHE_MAVEN_M2"  
    paths:  
      - .m2/repository
```

Pour garder en cache.
Ici les dépendances Maven

```
variables:  
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
```

```
stages:  
  - test  
  - build  
  - deploy
```

```
tester:  
  stage: test  
  script:  
    - mvn test
```

Pour définir quand on déclenche
le job. Détails ici:

https://docs.gitlab.com/ee/ci/jobs/job_control.html

```
java-build:  
  stage: build  
  rules:  
    - if: $CI_COMMIT_BRANCH == "main"  
  script:  
    - mvn install -DskipTests=true #this time we install because we want the artefacts to be installed in local rep  
artifacts:  
  name: "Maven artifacts"  
  paths:  
    - "**/target"
```

Déploiement automatique

Déploiement

Installer, configurer et déployer un logiciel ne consiste pas seulement à s'occuper du dit logiciel, mais aussi de son environnement (y compris sa pile d'exécution) et de ses interactions avec d'autres logiciels. On parle d'orchestration.

This layer is optional yet very classical



Challenge

Exemple : Déployer Fiware

1. Déployer MongoDB
 - a) Installer les libs de MongoDB
 - b) Configurer MongoDB pour qu'elle communique avec FIWARE
 - c) Démarrer MongoDB
 - d) Configurer les tables, users, etc.
2. Installer Orion Broker
 1. Installer les libs
 2. Configurer, y compris la communication avec Mongo
 3. Démarrer Orion
3. Plus ou moins la même chose pour QuantumLeap ...
4. Configurer la machine pour ouvrir les bons ports etc.



**Parfois compliqué.
Souvent long.
Une erreur et tout peut être à refaire
...**

Motivation

- 1. Répliquer / réutiliser / capitaliser des configurations, piles logicielles, déploiements :**
 - Rejouer des déploiements qui ont fait leurs preuves.
 - Garder le moyen de remettre en place l'infrastructure pour la version X du logiciel.

- 2. Automatiser des configurations / déploiements :**
 - Gagner du temps.
 - Déployer à large échelle.
 - Minimiser l'intervention humaine.

Paysage

- De nombreuses solutions existent :

- Ansible
- Puppet
- Chef
- Vagrant
- Cloudify
- Brooklyn
- ...



- Ils permettent:

- Installation, configuration, exécution, arrêt, *monitoring*, *mise à jour (parfois)*.
- Installation à distance : avec ou sans agent de déploiement.
- Inclus souvent le provisionnement des ressources Cloud / virtualisées.

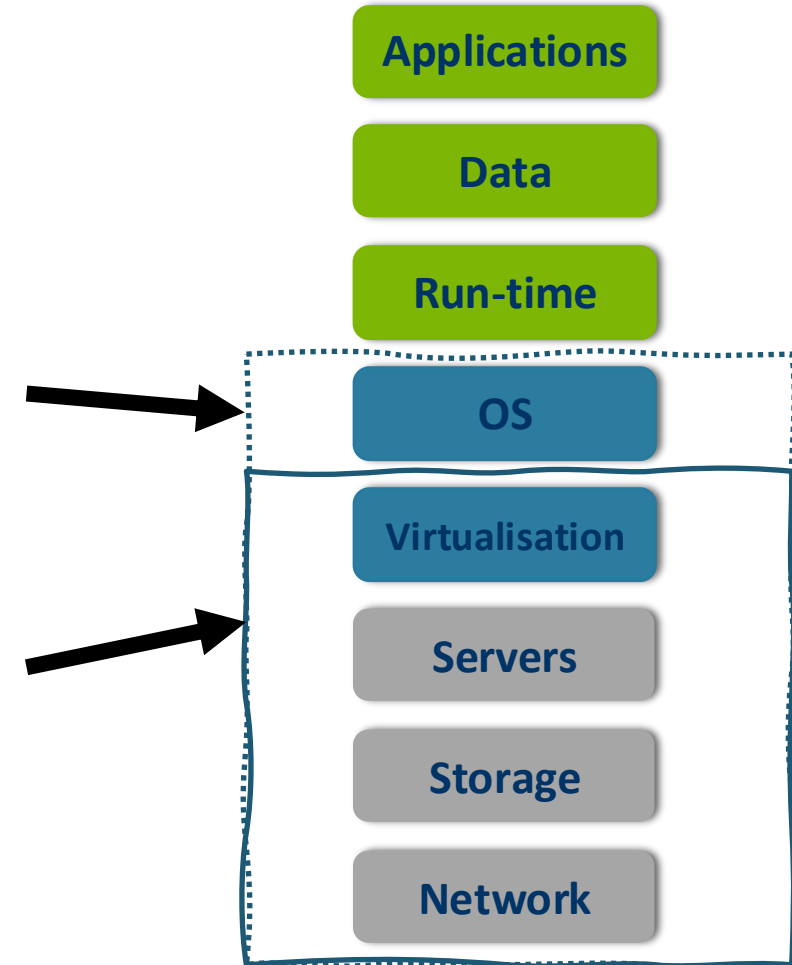
Grandes approches

- **White-box :**

- Pour chaque logiciel, instance de logiciel on refait son installation (et donc on peut la customiser).
- On décrit comment gérer le cycle de vie du logiciel via un/des scripts.
- On contrôle tout.

Si virtualisation, le choix de l'OS fait parti du processus de provisioning

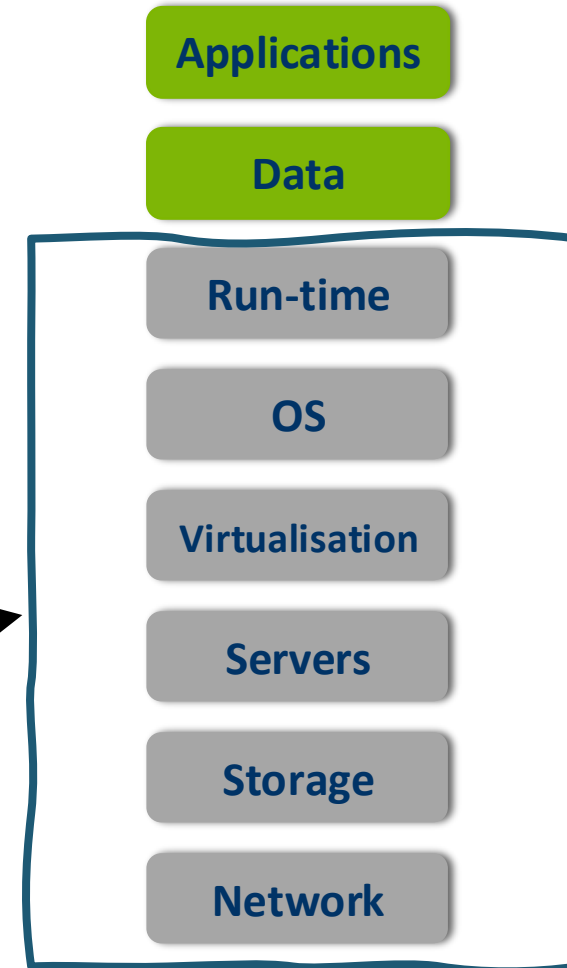
Sous le contrôle du provider quelqu'il soit



Grandes approches

- **White-box :**
 - Pour chaque logiciel, instance de logiciel on refait son installation.
 - On décrit comment gérer le cycle de vie du logiciel via un/des scripts.
 - On contrôle tout.
- **Black-box :**
 - Typiquement pour du PaaS (Platform as a service).
 - Un donne le binaire et la plateforme s'occupe de gérer son cycle de vie.
 - Peu ou pas de contrôle.

Sous le contrôle du
provider quelqu'il soit



Grandes approches

- **White-box :**

- Pour chaque logiciel, instance de logiciel on refait son installation.
- On décrit comment gérer le cycle de vie du logiciel via un/des scripts.
- On contrôle tout.

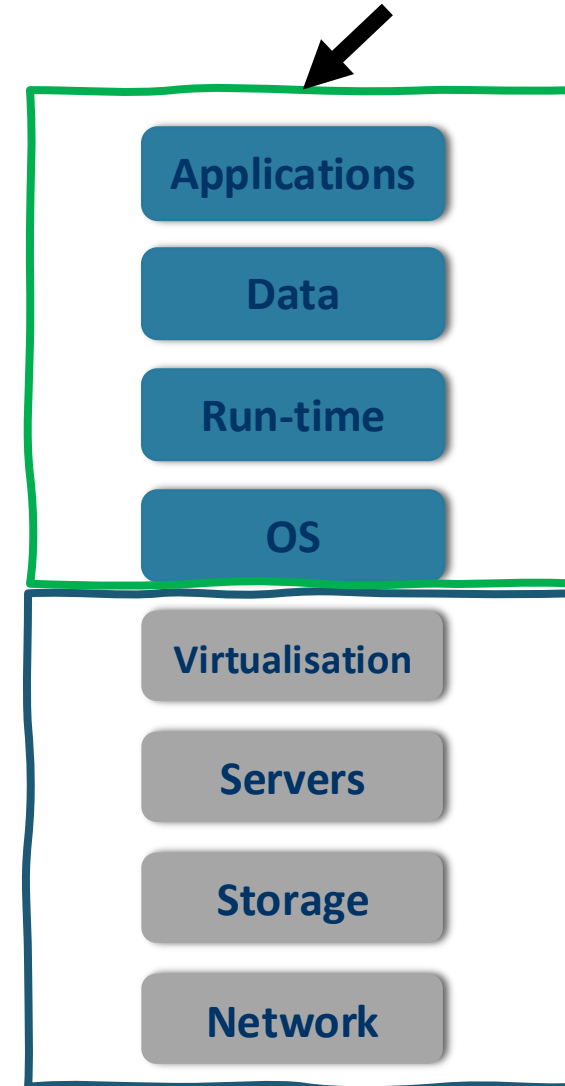
- **Black-box :**

- Typiquement pour du PaaS (Plateforme as a service).
- Un donne le binaire et la plateforme s'occupe de gérer son cycle de vie.
- Peu ou pas de contrôle.

- **Grey-box :**

- On exécute le déploiement d'une instance du logiciel que l'on capture dans une « image ».
- Les déploiement subséquent consistent simplement à créer des instance de l'image.
- On peut customiser par la suite chaque instance.

On fait une instance d'une image
(la limite avec l'OS n'est pas si claire)



Grey-box

- Nous nous intéressons plus particulièrement aux approches Grey-box.
- La **virtualisation** est un des « facilitateurs ».
 - Fourni le concept d'image.
 - Permet de déployer plusieurs applications à partir d'images sur une même machine hôte.

Virtualization

*« **Virtualization** refers to the act of creating a virtual (rather than actual) version of something, including virtual computer hardware platforms, storage devices, and computer network resources. »*

-- Wikipedia

- Exécuter plusieurs systèmes d'exploitation et/ou applications isolés sur un même hôte physique.
 - Lorsque l'on crée une ressource virtuelle on parle de « **provisionning** »
 - Le provisionning fait parti de l'activité de déploiement.

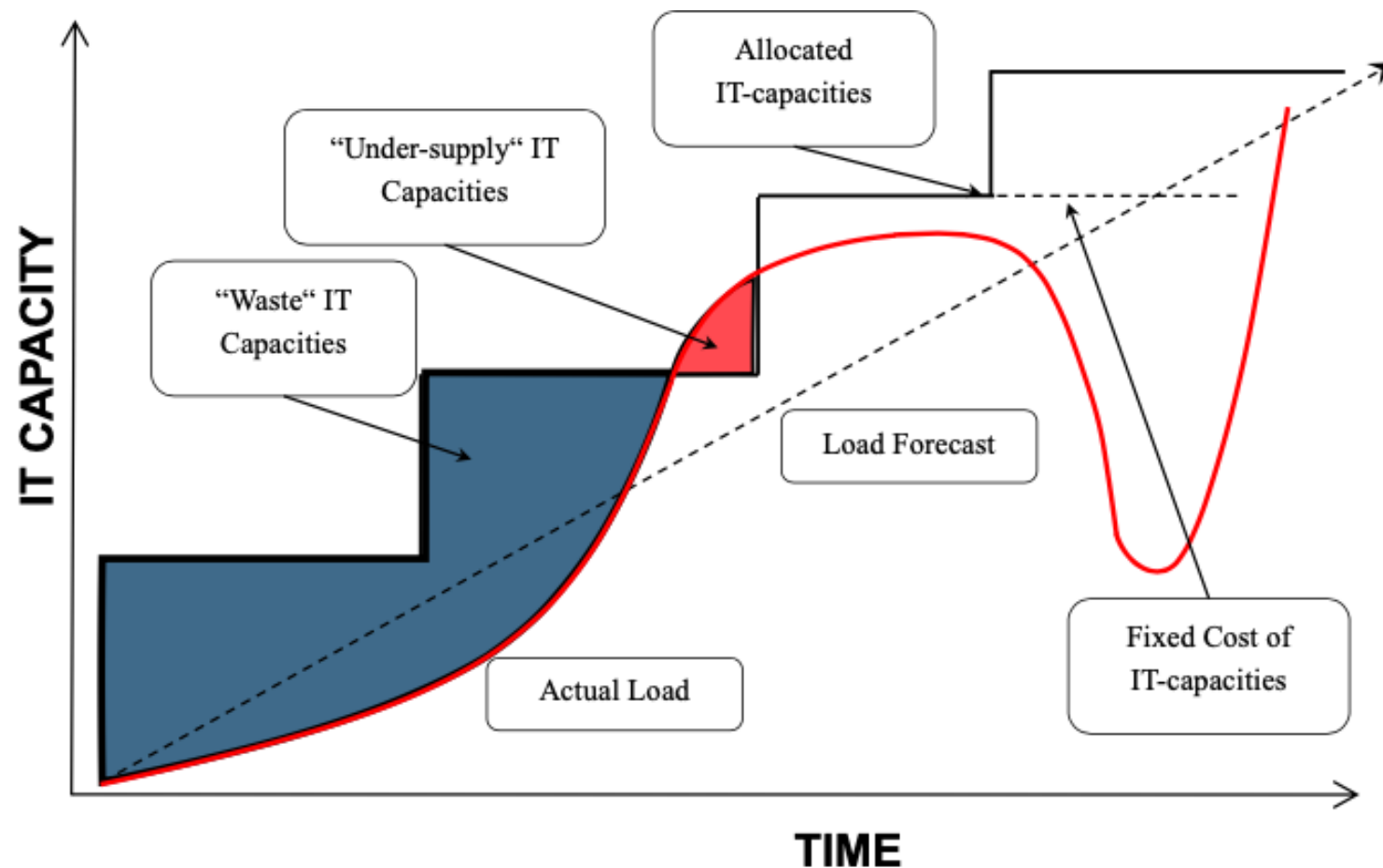


Cloud computing

- Cloud computing in short:
 - **Large-scale** and **accessible on demand resources**.
 - Network
 - Storage
 - Compute
 - Software
 - **Available** via Web service calls **through** the **Internet**.
 - Short- or long-term access on a **pay per use** basis.

Optimize IT Capacity to the load

Enabled by
virtualization !



Courtesy of Microsoft

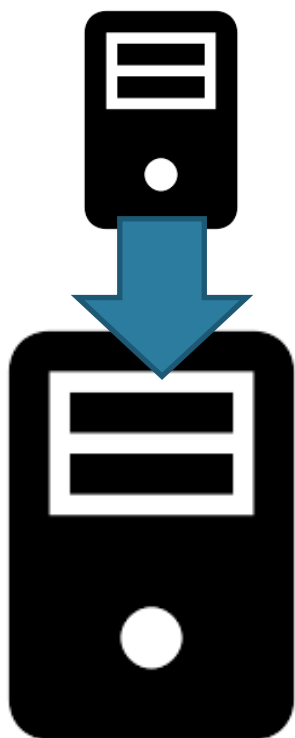
Elasticity and Scalability

- **Scalability:** the ability of a service to sustain variable workload while fulfilling quality of service (QoS) requirements, possibly by consuming a variable amount of underlying resources.
- **Elasticity:** the ability of a service to rapidly provision and deprovision underlying resources on the fly.

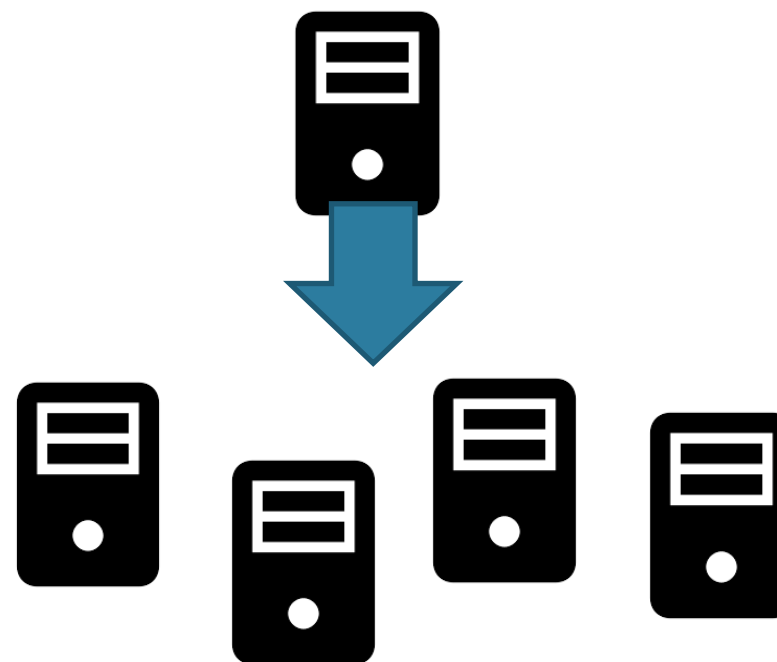
One does not guarantee the other!

Scalability

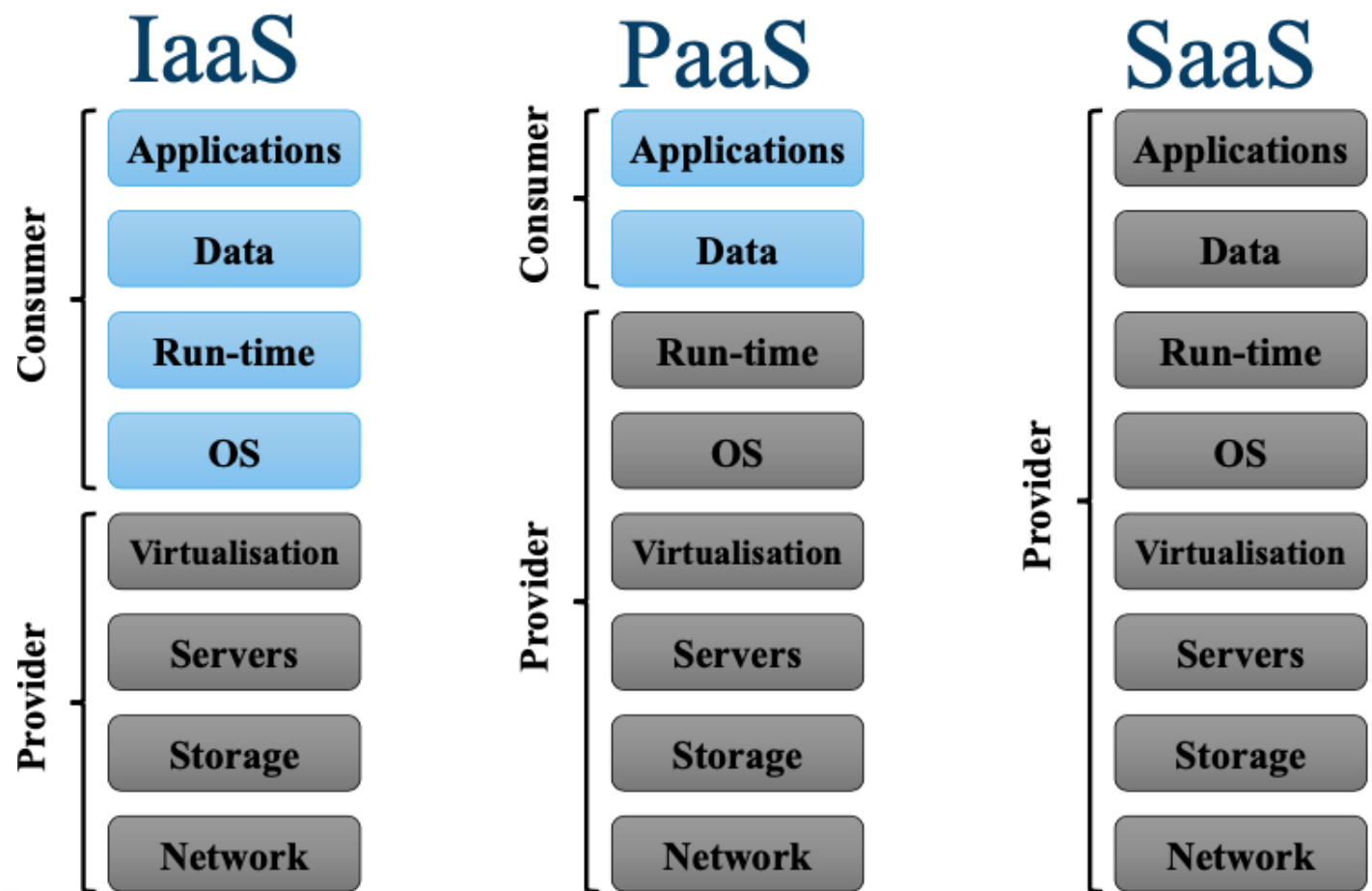
Vertical



Horizontal



The Cloud Computing Stack



Deployment model

- **Private Cloud**

- Owned by the organization. Said to be more secure as the storage and processing stays under the organization control.
- E.g., OpenStack, Cloud Foundry.

- **Public Cloud**

- Hosted at the provider premises, who is in charge of its maintenance and management.
- E.g., AWS, Azure.

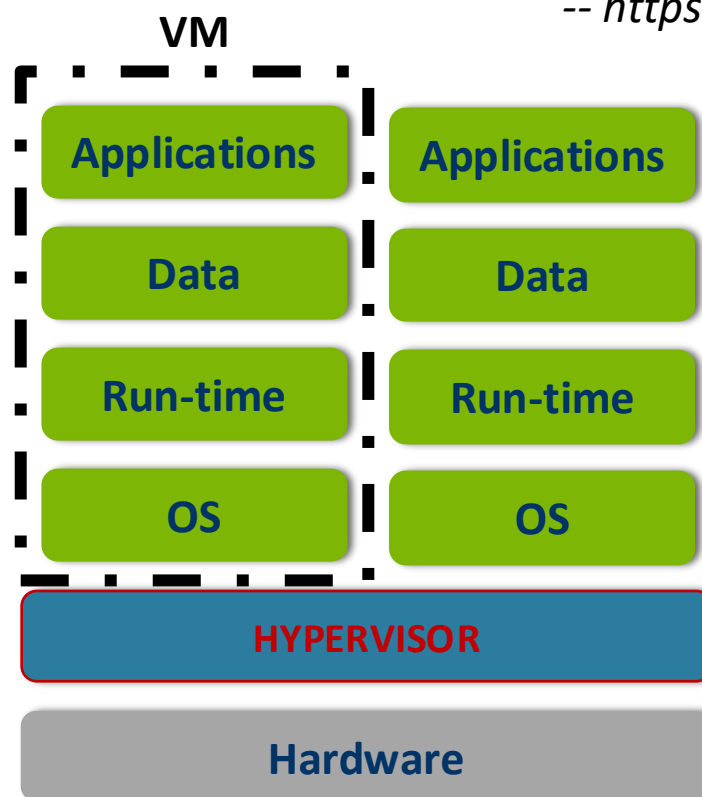
- **Hybrid Cloud**

- Composition of two or more public and private clouds.

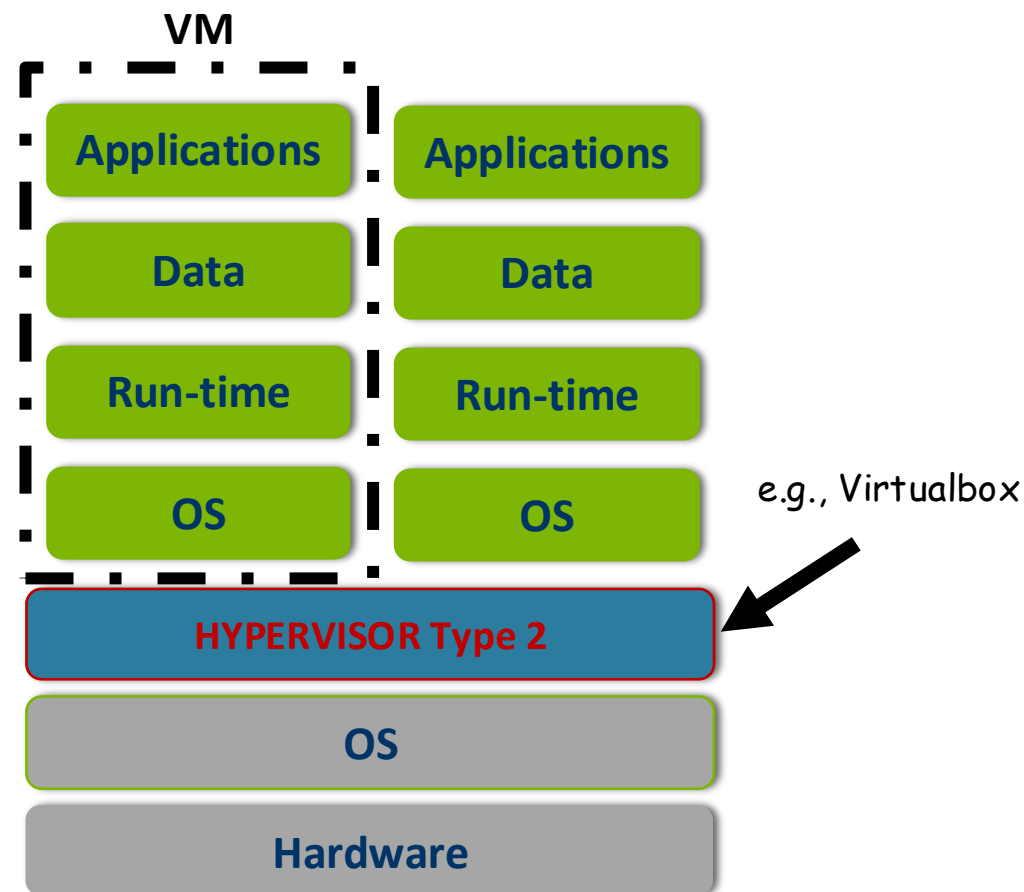
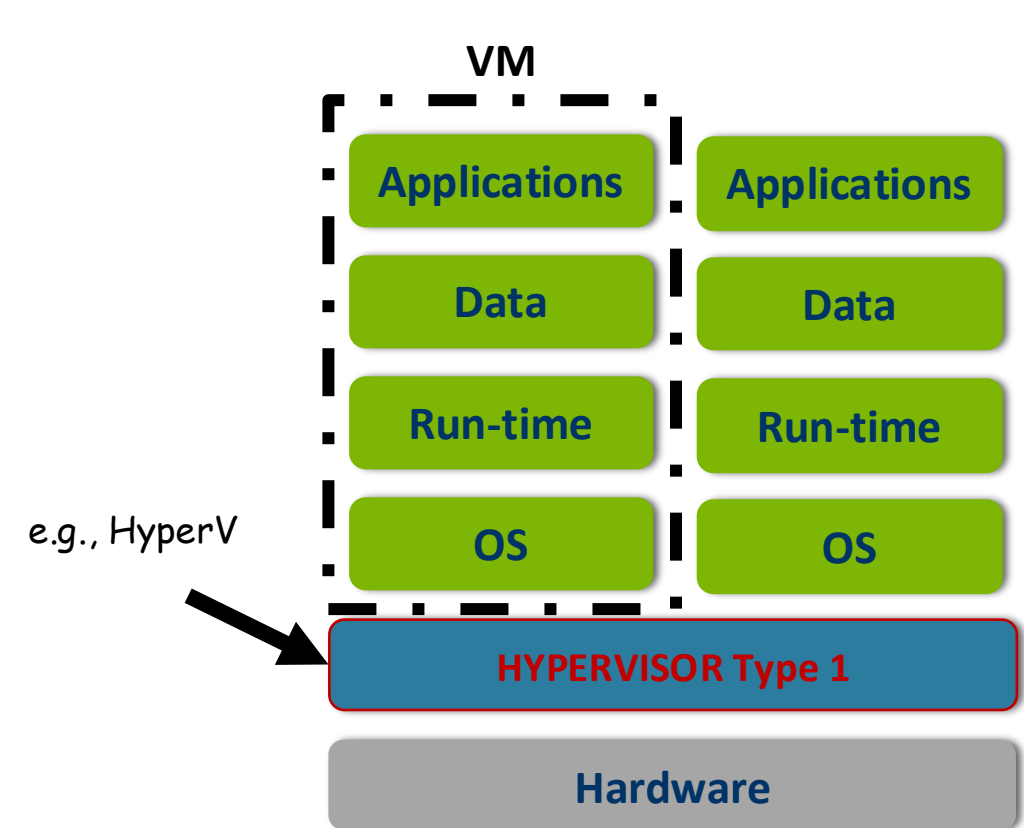
Virtualisation & hyperviseur

- **Hyperviseur** : « *The virtualization component that manages the guest OSs on a host and controls the flow of instructions between the guest OSs and the physical hardware.* »

-- <https://csrc.nist.gov/glossary/term/hypervisor>



Type d'hyperviseurs

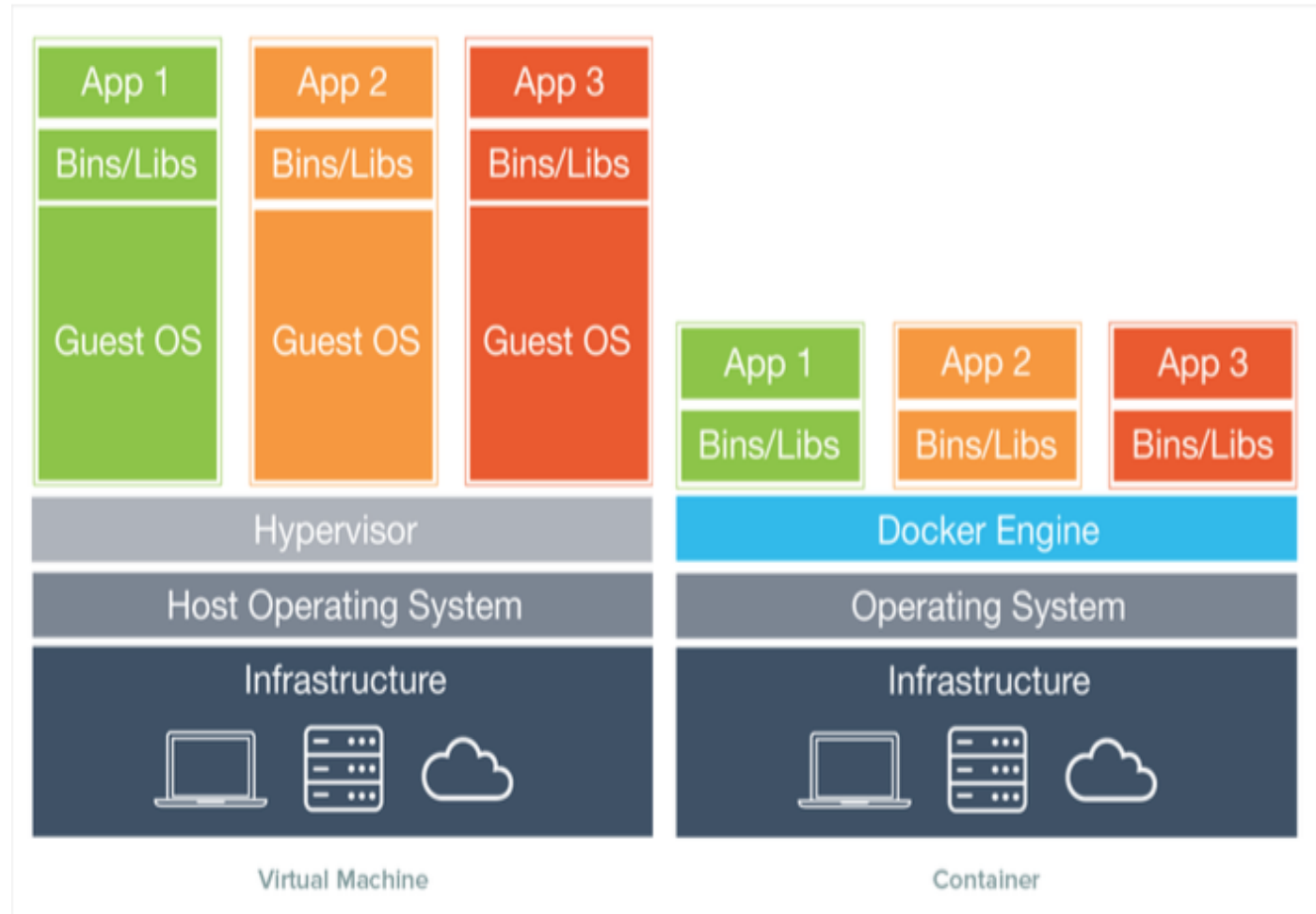


Conteneurisation

Approche grey-box

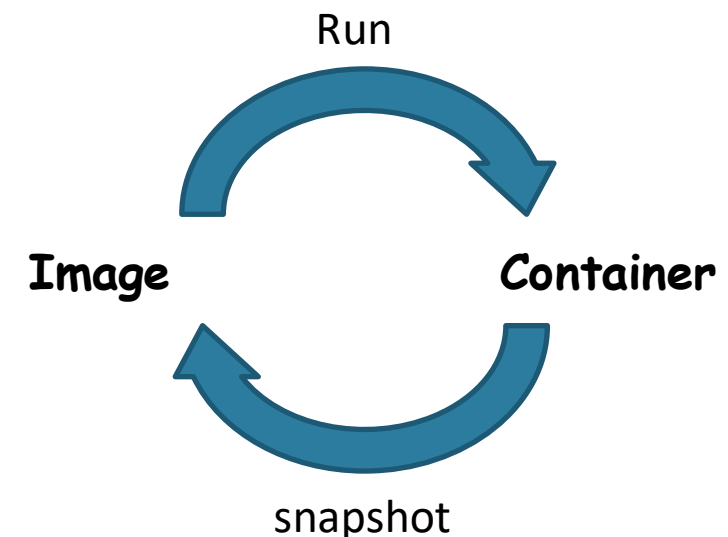
Conteneurisation

- Virtualisation **légère**.
 - Utilise l'OS sous-jacent pour les appels systèmes.
 - **Isolation** (réseau, application, librairies).



Containers Concepts

- **Image** : un instantané immuable du système de fichier (donc qui inclus l'application et ses « dépendances »).
- **Container:**
 - Un système de fichier temporaire:
 - Construit sur le système immuable de l'image.
 - Détruit lorsque le container est tué sauf si on crée une image avant cela.
 - Accessible en lecture et écriture.
 - Un groupe de processus.
 - Une pile réseau (172.17.x.x)



Isolation – Comment ?

- **4 ingrédients clés :**

1. **Namespaces** : Donner une vue sur des ressources partagées – i.e., grouper celles qui seront visible ou non pour un processus.
2. **CGroups** : Créer des groupes de processus ou tâches et applique des limitations sur l'utilisation du hardware (partage des ressources entre processus).
3. **(Layered) filesystems** : Création d'un système de fichier isolé.
4. **Seccomp** : Sécurité des appels systèmes.

Accessibles via des appels systèmes

Namespaces

- Dans un système UNIX, un certain nombre de valeurs sont communes à tout le système et identifie des ressources partagées:
 - le hostname est le même pour tout le monde ;
 - chaque processus a un PID unique ;
 - la configuration réseau (IP, ports ouverts) est commune à tous les processus sur le système ;
 - tout le monde voit les mêmes montages sur les mêmes répertoires.
- **Problème du point de vue du container** : si le système virtuel (invité) veut changer une de ces valeurs alors il la change pour tout le monde !
 - **E.g., le système invité ne peut pas changer son hostname sans changer celui de l'hôte**

Namespaces à la rescousse !

- Un namespace est un regroupement de processus avec une vision commune d'une ressource partagée.
 - On passe d'une vision commune pour tous à ...
 - ... une vision par groupe de processus.
- Ce qui permet de créer pour nos containers leurs propres environnements virtuel – *i.e., comme si un container avait son propre environnement*:
 - Donne une vue et permet de partager des ressources globales.
 - Types de namespaces : PID, mount, network, IPC, user, UTS, time, cgroup

Type de namespace

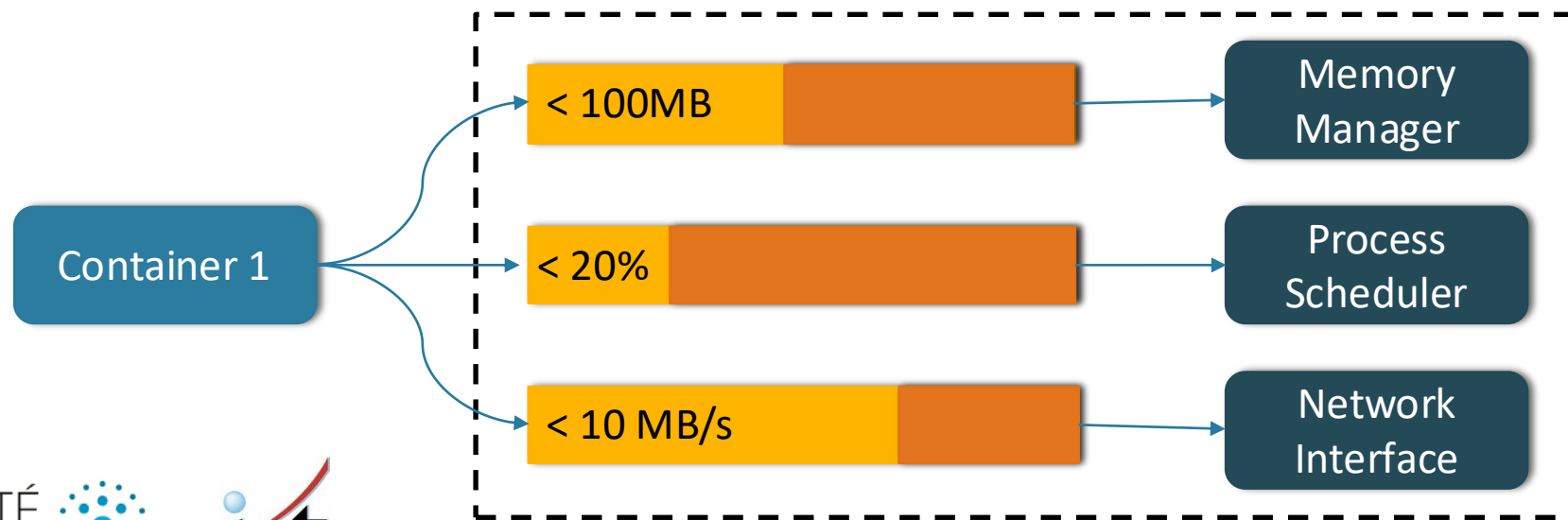
- **PID** : Vue sur les processus et leurs enfants dans le système. Si processus dans le namespace alors accessible sinon on considère qu'il n'existe pas. Le premier process dans le namespace à le PID 1 (comme le premier process dans votre OS).
- **Mount** : Les processus dans ce namespace ont leur propre table de montage. Du coup combiné avec « pivot_root » ou « chroot » permet de définir son propre système de fichier !
- **Network** : Les processus dans ce namespace ont accès à leur propre pile réseau. Des containers peuvent être dans le même réseau « virtuel » et malgré tout faire le mapping avec la vrai interfaces réseaux de la machine hôte.

Types de namespace

- **IPC** : Pour isoler les communications inter-processus.
- **user** : Permet aux processus dans le namespace d'avoir leur propres user ID et permet de faire le mapping avec les uids de l'hôte. Typiquement ce qui permet de simuler un accès root dans un container sans être root sur la machine hôte !
- **UTS** : Permet aux processus dans le namespace d'avoir leur propre hostname.
- **Time** : Permet aux processus dans le namespace d'avoir leur propre clock.

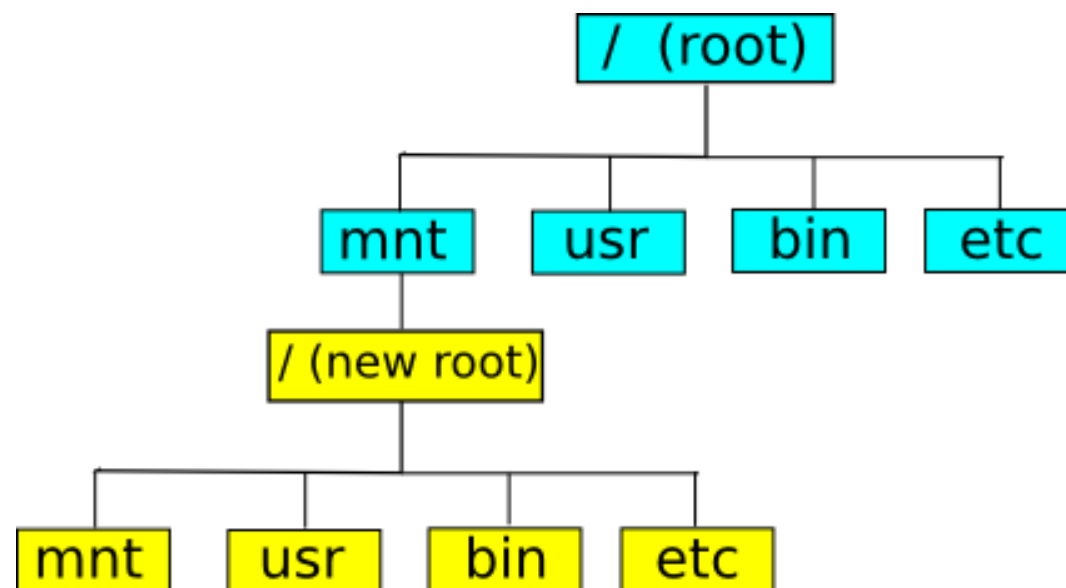
Control Groups

- Permet entre autres de :
 - Limiter les ressources utilisées par un container - i.e., ce que les processus du container peuvent utiliser.
 - Prioritiser les accès aux ressources entre groupes (plus de bande passante, plus de cpu).
 - Créer un point de sauvegarde et redémarrer un groupe.



Filesystem

- Le container doit avoir son propre système de fichier (et donc son propre environnement), lorsque l'on crée un container on :
 - Place l'environnement (le filesystem) souhaité dans un répertoire.
E.g., debootstrap permet d'installer un système debian dans un sous-répertoire
 - Change la racine du système de fichier pour ce container en choisissant le répertoire cité ci-dessus. Et donne accès qu'à une partie du système.
E.g., avec pivot_root ou chroot



Et si on faisait un container ? (live demo)

1. Création du système de fichier (un ubuntu)

```
debootstrap focal ./ubuntu-rootfs http://archive.ubuntu.com/ubuntu/
```

2. Changer la racine pour le container

```
mount -t proc none ./ubuntu-rootfs/proc  
mount -t sysfs none ./ubuntu-rootfs/sys  
mount -o bind /dev ./ubuntu-rootfs/dev  
chroot ./ubuntu-rootfs /bin/bash
```

} Pour accéder aux infos essentielles
du système hôte !

Ok, filesystem isolé mais tout le monde voit tout le monde !
- e.g., je peux kill un process du parent !

Et si on faisait un container ?

3. Création des namespaces pour notre container

```
unshare -p -f -ipc --mount-proc=$PWD/ubuntu-rootfs/proc --ipc  
\ chroot ./ubuntu-rootfs /bin/bash
```

Look hostname
Look at top

4. Configuration du cgroup

```
#Cgroup pour le container1  
mkdir /sys/fs/cgroup/cpu/container1  
#definition quota CPU  
echo 100000 > /sys/fs/cgroup/cpu/container1/cpu.cfs_quota_us  
#On s'assure de repartir d'un cgroup vide  
echo 0 > /sys/fs/cgroup/cpu/container1/tasks  
#On met le process courant dans le cgroup  
echo $$ > /sys/fs/cgroup/cpu/container1/tasks
```

Outil de gestion de containers

- **Besoin d'outils pour créer, arrêter, démarrer, monitorer, composer, etc. des containers.**
 - Docker : construit sur runC (Open Container Initiative - OCI).
 - Podman : open source, compatible Docker, sans daemon.
 - LXC : un des premiers, bas niveau.
 - ...

Docker

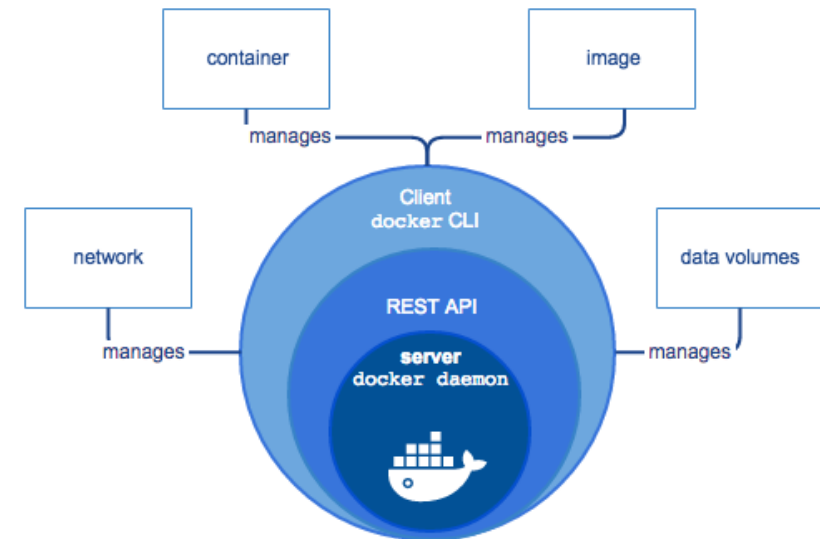
Approche grey-box - Docker

C'est quoi

- Une mécanisme de containerisation et de gestion de container :
 - Virtualisation **légère** avec des images **immutable** et légères déployable très **rapidement**.
- **?!!**
- Docker permet d'exécuter vos applications dans des containers isolés et connectés via des réseaux virtuels.
 - Les containers sont réutilisables et portables.
 - Le cycle de vie des containers peut être géré dynamiquement.

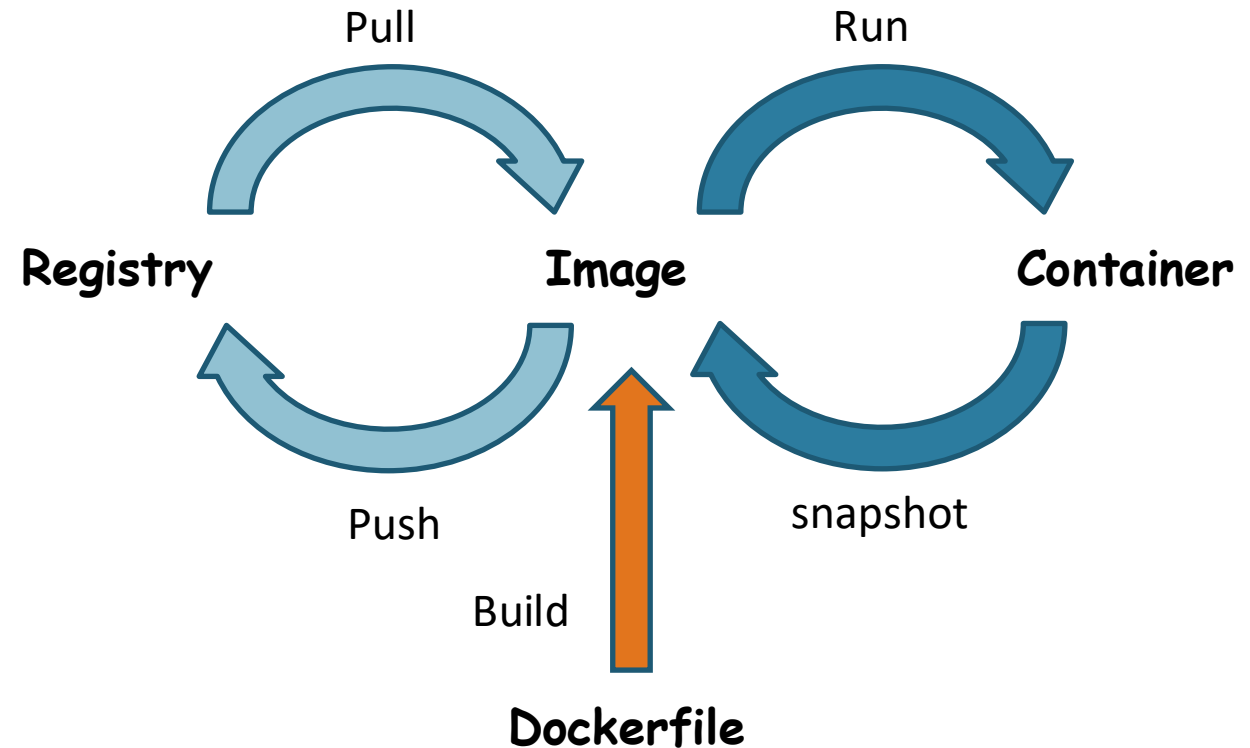
C'est aussi

- Un moteur d'exécution pour gérer le cycle de vie des containers et construire des images
- Des APIs :
 - Pour gérer, publier des images
 - Construire des images
 - Gérer des containers
 - Composer des containers
 - Passer à l'échelle (gérer des cluster d'images)

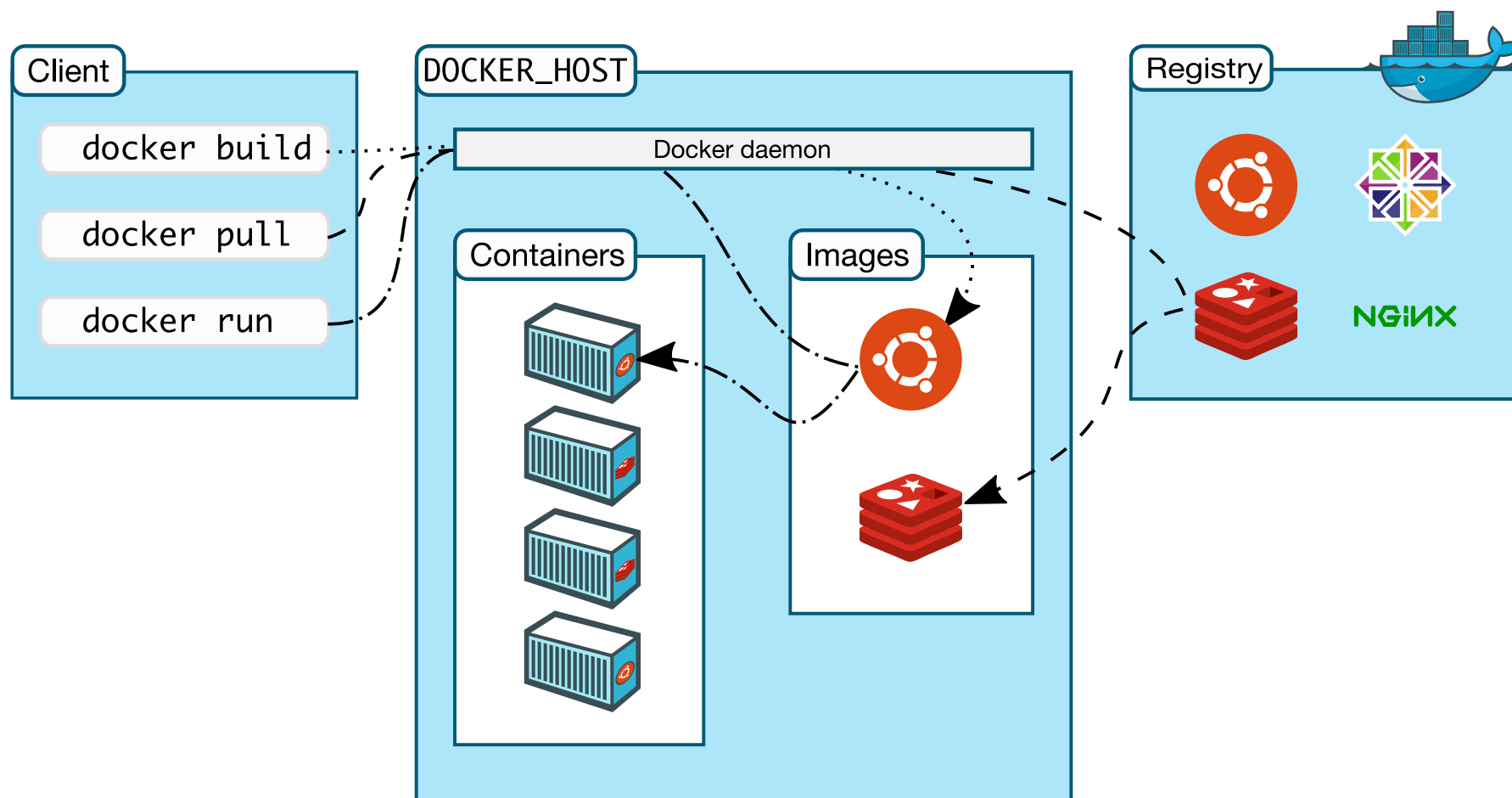


Principaux éléments manipulés

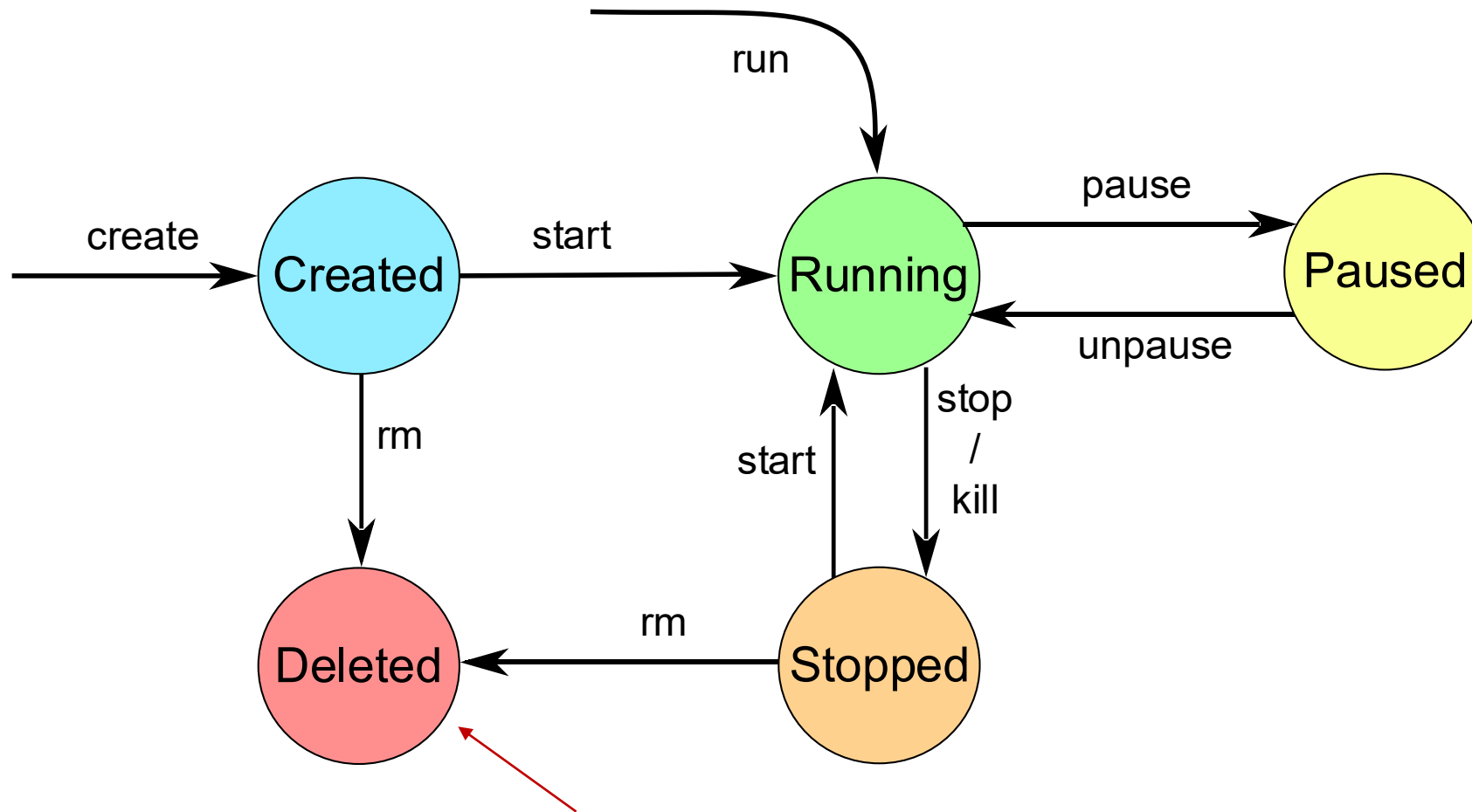
1. **Container**
(instance, en général
1 application/container)
2. **Image**
(niveau meta)
3. **Registry**
(Registre d'images)
4. **Dockerfile**
(niveau meta-meta!
Contruction image)



Architecture



Cycle de vie d'un container



Les données ajoutées dans le container après son démarrage sont perdues au delete

Quelques commandes

- Lister les containers existant

```
docker ps -a
```

- Créer un container a partir d'une image

```
docker create <image>
```

← Système de fichier initialisé
à partir de l'image

- Créé (si pas déjà fait) et démarre un container

```
docker run <image>
```

← -d detached mode

- Démarrer un container

```
docker start <container>
```

← -v /hostpath:/ctrpath

- Arrêter un container

```
docker stop <container>
```

- Tuer un container

```
docker kill <container>
```

Quelques commandes

- Supprimer un container
`docker rm <container>`
- Créer une image a partir d'un container
`docker commit <container> <image>`
- Exécuter une commande dans le container
`docker exec <container> args`
- Accéder aux logs d'un container
`docker logs -f <container>`

← Système de fichier terminé
(persistent entre start et stop)

Redémarrage automatique

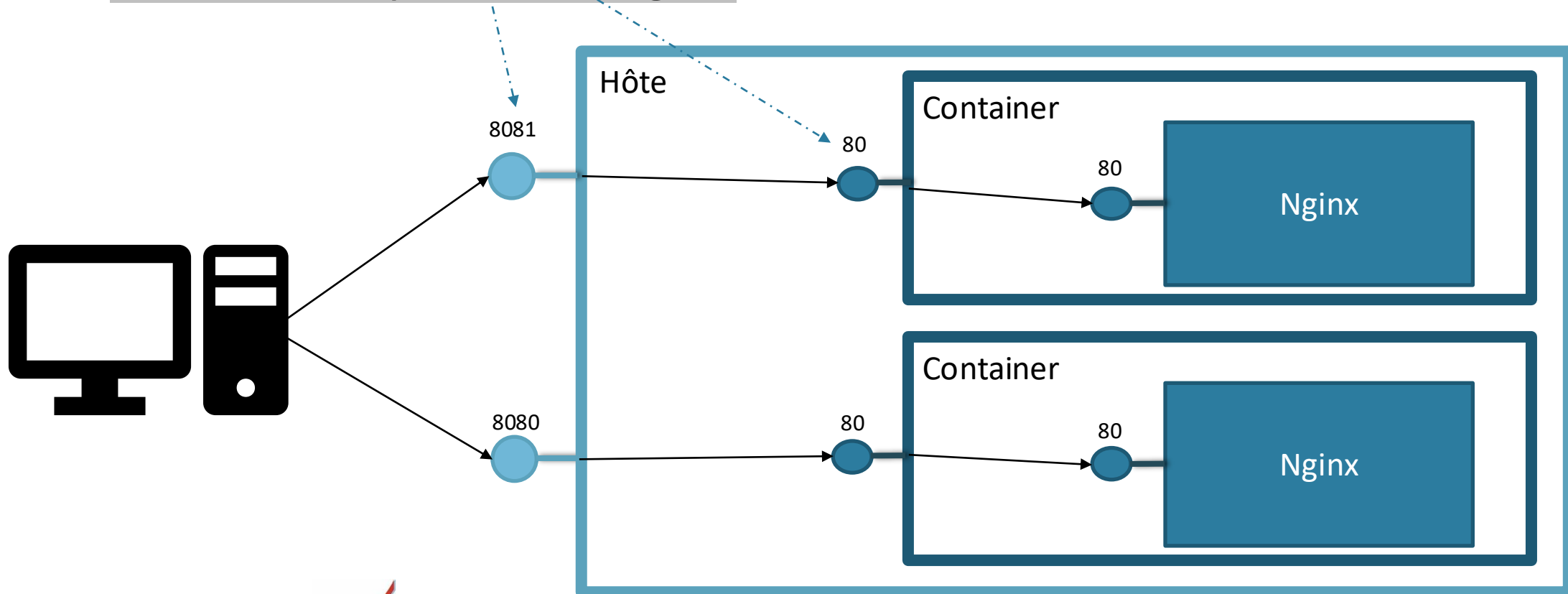
- Docker propose des mécanismes de redémarrage automatique d'un container.
- Démarrer le container avec l'option `--restart` avec pour argument :
 - `no` : valeur par défaut.
 - `on-failure` : redémarrer si container « terminated/stopped » sur erreur.
 - `Always` : redémarrer si « terminated / stopped ».
 - `unless-stopped` : redémarrer si « terminated ».

Réseau et gestion des ports

- Par défaut, les containers ne sont pas accessible depuis l'extérieur !
- Mais on peut **créer un réseau de containers** capable de communiquer entre eux ...
- ... et **exposer un container** au monde extérieur (i.e., des applis qui ne sont pas nécessaire sous docker) via la **machine hôte**.
- Tout cela se fait :
 - Via namespaces
 - Via iptables

Gestion des ports

- Il est possible d'exposer des ports au **lancement** d'un container
`docker run -p 8081:80 nginx`



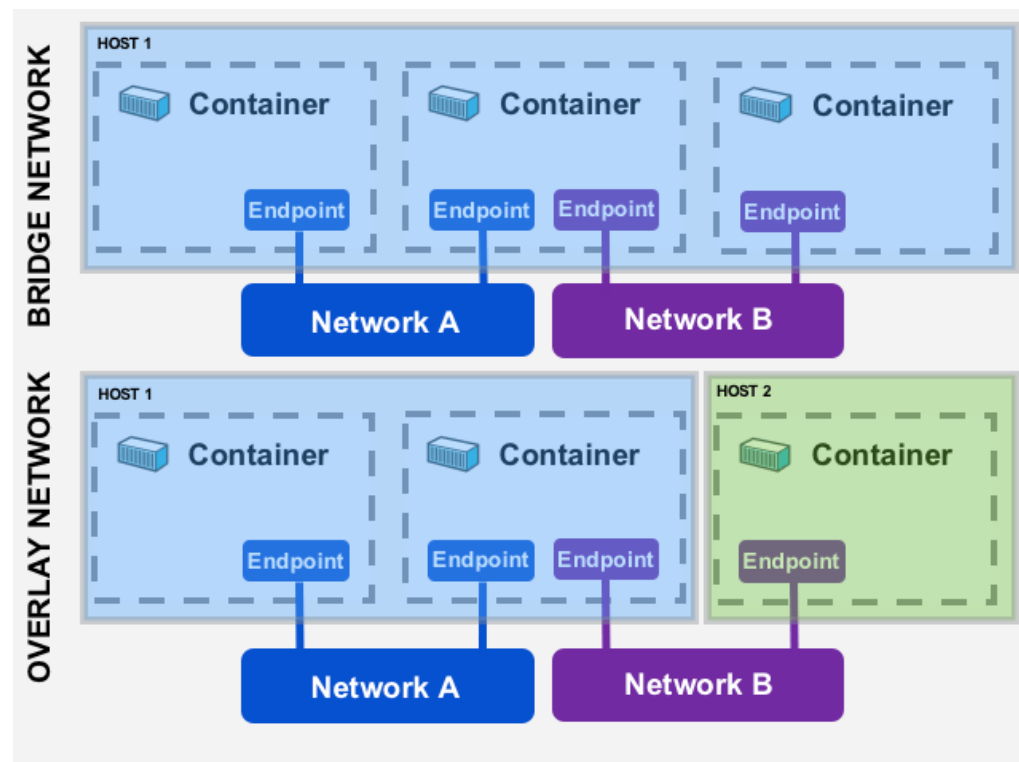
Réseau

- Le contrôleur de réseaux dans la docker engine repose sur des pilotes.
- Un pilote contrôle un réseau et le gère entièrement.
- Docker vient nativement avec 6 pilotes :
 - **Host** : the container uses the Host's network stack, without any namespace separation, and while sharing all of the host's interfaces.
 - **Bridge** : creates a Docker-managed Linux bridge on the Docker host. By default, all containers created on the same bridge can talk to each other.
 - **Overlay** : creates an overlay network that may span over multiple Docker hosts. It uses both local Linux bridges and VXLAN to overlay inter-container communication over physical networks.
 - **MACVLAN** : uses the MACVLAN bridge mode to establish connections between container interfaces and parent host interfaces.
 - **None** : gives a container its own network stack and namespace, without any interfaces. Therefore, it stays isolated from every other Network.

Bridge & Overlay

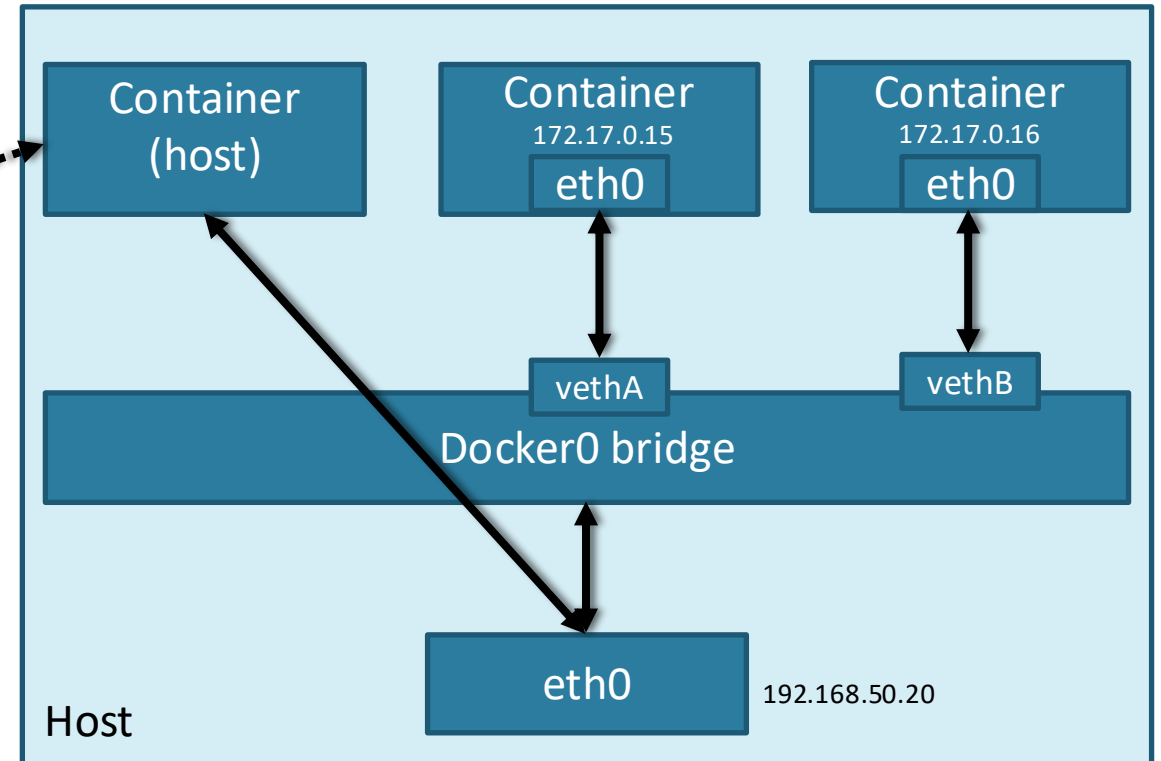
bridge permet de fournir un réseau, 172.17.0.0/16 par défaut, sur lequel seront connectés les conteneurs, ainsi qu'une passerelle par défaut, 172.17.0.1, gérée par l'hôte.

Les réseaux **overlay** sont des réseaux étendus entre les différentes machines formant le cluster Docker



Réseau de type : Host

la même adresse IP que l'hôte. Sous macOS ou Windows avec Docker Desktop, c'est l'adresse IP de la mini VM Linux qui porte la partie Docker !



Quelques commandes réseau

- Connaitre la liste des réseaux :
`docker network list`
- Supprimer les réseaux existants :
`docker network prune`
- Obtenir des détails sur un réseau :
`docker network inspect [nom_du_réseau]`
- Connecter un container à un réseau :
`docker network connect [nom_du_réseau] [nom_du_container]`
- Déconnecter un container d'un réseau :
`docker network disconnect [nom_du_réseau] [nom_du_container]`

Images

- Un fichier **immutable** (sorte de template) contenant le nécessaire pour créer un container.
 - Contient un système de fichiers (multi-layers).
 - Une image est généralement constituée d'un ensemble de layers.
 - Un layer stock les informations par rapport à l'image sur laquelle il se base.
 - Pour construire les images de manière incrémentale.
(on ne reconstruit que ce qui est nécessaire)
 - Pour réduire le volume de ce qui doit être téléchargé lorsque l'on pull une image.
(on ne télécharge que ce qui n'est pas disponible en local)

➤ **Un layer est une image !**

Images & tag

- Chaque image (donc chaque layer) est identifié par un tag.
 - Auto-généré
 - Séquence de lettre et de chiffre que vous voyez en faisant un `docker ps aux`
 - Défini à la main sous le format :
 - `<repository>/<image_name>:<tag>`
 - Ou pour le images officielles : `<image_name>:<tag>`

Creation d'images

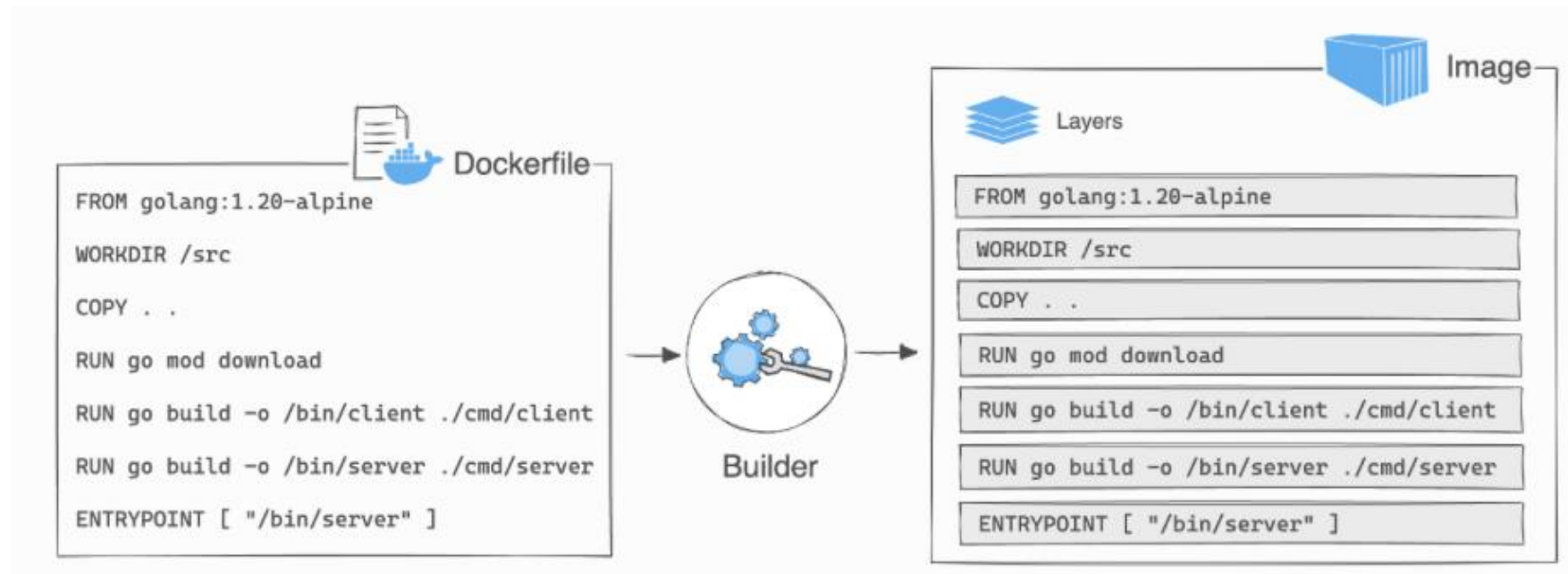
1. Commit d'un container.

2. Docker builder :

1. Un langage pour décrire comment construire une image.
<https://docs.docker.com/reference/builder/>
2. Un mécanisme pour construire l'image comme une succession couches.
(avec un cache pour les layers n'ayant pas changés)
`docker build -t <tag> <Dockerfile>`
3. Prend en entrée un fichier Dockerfile et à accès aux fichiers dans le répertoire ou se situe le fichier Dockerfile.

Dockerfile

- Un fichier contenant un ensemble **ordonné** d'instructions.
 - Les instructions sont traitées dans l'ordre séquentiellement ...
 - ... et chaque instruction consiste en un layer de l'image à construire.



Dockerfile

- Les instructions ont la forme :
 - INSTRUCTION arguments
- Commence toujours par FROM
 - On construit à partir d'une image existante
 - Peut être scratch (une image vide !)
- Si une instruction échoue, les précédents layers construits par les précédentes instructions sont gardés en cache.

Layers	Cache?
FROM golang:1.20-alpine	✓
WORKDIR /src	✓
COPY . .	✗
RUN go mod download	✗
RUN go build -o /bin/client ./cmd/client	✗
RUN go build -o /bin/server ./cmd/server	✗
ENTRYPOINT ["/bin/server"]	✗

Dockerfile

```
# Couche 1 = A partir de cette image d'ubuntu
FROM ubuntu:18.04

# Couche 2 = On exporte une variable d'environnement
ENV APP_HOST='192.168.1.23'

# Couche 3 = On copie le répertoire qui se trouve au même emplacement que Dockerfile dans app
COPY . /app

# Couche 4 = « on se place dans le répertoire » app. Les commandes suivantes seront relatives à ce répertoire
WORKDIR /app

# Couche 5 = On exécute une commande pour construire l'application
RUN make . #ou RUN ['make', '.']

# Couche 6 = Commande à exécuter au démarrage du container
CMD python ./app.py #ou CMD ['python', './app.py']
```

Autres instructions pratiques

- `EXPOSE port`
 - **Documente** le port sur lequel devrait écouter le container
- `HEALTHCHECK [options] CMD <command>`
 - Instruction décrivant comment **vérifier l'état** du container en exécutant une commande à l'intérieur du container
- `VOLUME ['path']`
 - Pour créer un point de montage

Quelques astuces

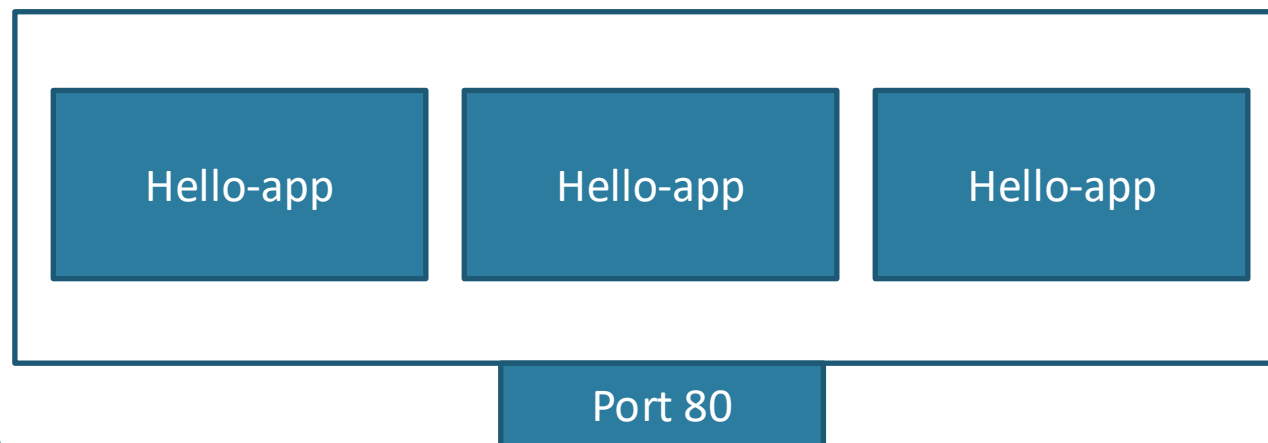
- Il est possible de déclencher plusieurs commandes bash dans une instructions (parfois on a pas envie de faire un layer par commande).
- **Attention à la taille de vos images. A optimiser !**
 - Partir d'une image petite (e.g., alpine approx. 3Mo)
 - Si plusieurs containers, réutiliser un maximum de layers
 - Ne pas inclure des fichiers non nécessaires à l'application (e.g., le dockerfile lui-même, souvent les sources !)
 - Ne pas construire l'application dans le container ! On construit l'application et on copie le résultat de la construction dans le container.

Orchestration de containers

- **Docker compose** offre un langage permettant de définir :
 - Comment déployer et configurer les containers.
 - Les dépendances entre containers (A doit être déployé avant B).
 - Les relations et le(s) réseau(x) entre containers.
- Définit dans un fichier avec la syntaxe YAML.

Docker-compose

- Avec Docker-compose on déploie des applications multi-containers.
- Une application est en réalité composée de multiple **services**.
 - **Service = n containers** s'exécutant avec la même configuration et accessible via le même port.
 - Permet de répliquer une instance d'une application (scalabilité horizontale).



Ecrire un script pour Docker-compose

- Exemple simple à deux services : web et db.

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
  db:
    image: postgres
    ports:
      - "8001:5432"
```

On construit l'image du container à partir du Dockerfile se trouvant dans le répertoire courant. On lancera le container à partir de cette image

Mapping de port comme avec l'option -p

On lancera le container à partir de l'image déjà existante postgres

- On lance le déploiement des deux services avec la commande :
`docker compose up`
- On arrête le déploiement avec :
`docker compose down`

Compose et réseau

```
version: "3.9"
services:
  web:
    build: .
    ports:
      - "8000:8000"
  db:
    image: postgres
    ports:
      - "8001:5432"
```

Dans le code de web je peux communiquer avec db en utilisant l'URL postgres://db:5432

-i.e., pas besoin de connaître les IPs des containers !

When you run `docker-compose up`, the following happens:

1. A network called `myapp_default` is created.
2. A container is created using `web`'s configuration. It joins the network `myapp_default` under the name `web`.
3. A container is created using `db`'s configuration. It joins the network `myapp_default` under the name `db`.

Redéploiement

- **Ne reconstruit les images que si nécessaire**
 - Compose met en cache la configuration utilisée pour créer un container. Si le service n'a pas changé, compose réutilise le container existant.
- **Conserve les volumes.**
 - Si utilisation d'un volume par un service, si redémarrage ou mise à jour d'un service alors le volume est conservé.

Un exemple plus complet

```
mongo-db:
  image: mongo:3.6
  hostname: mongodb
  container_name: db-mongo
  expose:
    - "27017"
  ports:
    - "27017:27017"
  command: --bind_ip_all --smallfiles
  volumes:
    - mongo-db:/data
```

On démarre le container à partir d'une image

Pour spécifier un hostname autre que le nom du service

Pour spécifier le nom du container

Port accessible dans le réseau virtuel mais pas de l'extérieur

Mapping de port

Commande à exécuter au démarrage du container

Volume à monter. <path_hôte>:<path_container>

Dépendances

services:

web:

build: .

depends_on:

- db
- redis

redis:

image: redis

db:

image: postgres

web dépend de redis et db.

L'instruction `depends_on` indique que web sera démarré **uniquement** lorsque db et redis auront eux-mêmes **démarrés**.

Attention ! Un container peut être démarré avant que l'application qu'il exécute soit prête à être utilisée

Dépendances & Healthcheck

- Healthcheck permet vérifier le statut d'une application tournant dans un container.

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost"]
  interval: 1m30s
  timeout: 10s
  retries: 3
  start_period: 40s
  start_interval: 5s
```

- Utilisé par `depends_on` pour qu'il ne se base pas uniquement sur le statut du container.

```
services:
  web:
    build: .
    depends_on:
      db:
        condition: service_healthy
        restart: true
      redis:
        condition: service_started
  redis:
    image: redis
  db:
    image: postgres
```

Ici, compose attends le résultat positif du healthcheck de db avant de démarrer web

Networks

```
services:
  frontend:
    image: awesome/webapp
    networks:
      - front-tier
      - back-tier

  monitoring:
    image: awesome/monitoring
    networks:
      - admin

  backend:
    image: awesome/backend
    networks:
      back-tier:
        aliases:
          - database
      admin:
        aliases:
          - mysql

networks:
  front-tier:
  back-tier:
  admin:
```

- Dans cet exemple :
 - frontend appartient à deux réseaux (front-tier et back-tier)
 - monitoring appartient à admin
 - backend appartient à back-tier (et à le nom de database dans ce réseau) et à admin (avec le nom mysql)

Networks

```
networks:
  default:
    external:
      name: myExistingNetwork
  bridge_networks_1:
    driver: bridge
  bridge_networks_2:
    driver: bridge
  bridge_networks_3:
    driver: bridge
```

Par défaut nous containers sont dans le réseau déjà existant myExistingNetwork

On peut spécifier le type de réseau, typiquement bridge ou overlay !

Répliquations & restrictions ressource

```
services:
  frontend:
    image: awesome/webapp
    deploy:
      mode: replicated
      replicas: 6
```

```
services:
  frontend:
    image: awesome/webapp
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 50M
          pids: 1
        reservations:
          cpus: '0.25'
          memory: 20M
```

Docker et CI/CD

- La plupart des outils de construction automatique exécutent leur pipeline dans un environnement virtualisé:
 - VMs (e.g., travis)
 - Containers (e.g., GitLab CI)
- Du coup, lorsque l'on essaie de lancer docker dans un pipeline:
 - Docker dans une VM
 - Docker dans un container docker
 - *Attention à la configuration réseau ! Attention aux performances !*

Pourquoi utiliser Docker dans un pipeline?

- De nombreuses motivations parmi lesquelles :
 - Publier des images Docker que l'on aurait construit automatiquement.
 - Lancer des tests d'intégration !
 - Tester l'exécution distribuée !
 - Déployer !
- Ca donne quoi dans GitLab CI?

```
build_docker:
  stage: deploy
  tags:
    - dind
  image: docker:latest
  services:
    - name: docker:dind
  variables:
    DOCKER_TLS_CERTDIR: ""
```

On va faire du docker in docker (gitlab IUT)

On part d'une image avec docker

On active docker in docker

On désactive la partie TLS dans Certains cas

Déployer à distance

1. Définir la docker engine sur laquelle on veut envoyer nos commandes (REST)

```
export DOCKER_HOST="ssh://user@remotehost"  
docker-compose up -d
```

2. Utiliser les contextes pour passer d'une cible à une autre

```
docker context create remote --docker "host=ssh://user@remotemachine"  
docker context ls #pour lister les contextes  
docker --context remote ps  
docker-compose --context remote up -d
```



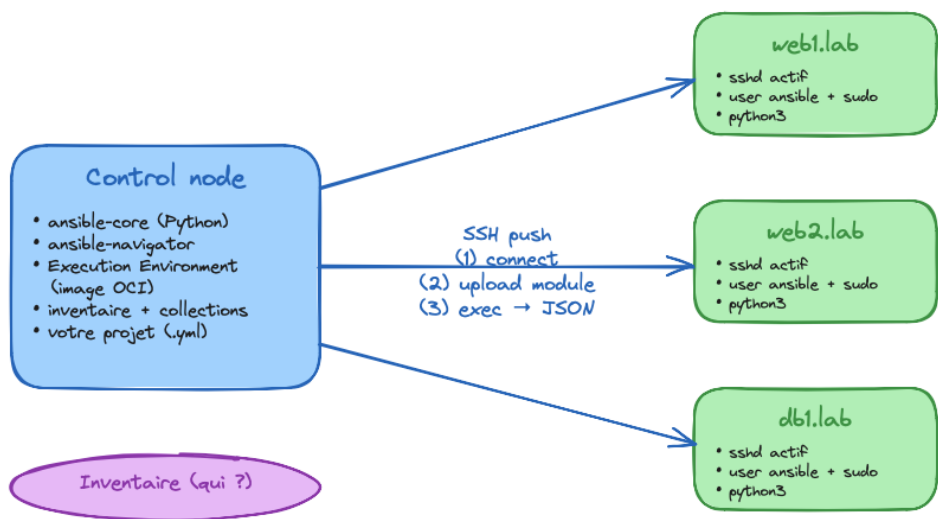
A N S I B L E

Configuration management - Ansible

Configuration Management System

- Ansible est un outil adapté pour la **configuration de systèmes**.
 - On est plus bas niveau que Docker-compose ou Kubernetes.
 - Objectif différent comparé à Docker build.
- Outil boîte blanche mais avec **idempotence (& diff)**.
- Outil assez léger avec peu de dépendances.
- Open source, écrit en Python

Pas d'agent vs agent



- **Un contrôleur sur lequel Ansible est installé :**
 - héberge ansible-core, ansible-navigator, l'inventaire et les collections.
 - Il pilote N machines distantes par SSH.
- **Des machines distantes :**
 - Sans agent (uniquement besoin de Python).
 - Gérées à travers SSH.

Approche déclarative

- On décrit **l'état souhaité du système**, on n'enchaîne pas des actions.
- Ansible détermine comment atteindre cet état. En faisant le moins de modifications possibles
 - i.e., on ne change que ce qui a besoin d'être changé
- Pour aller vers le principe d'idempotence : $f(x) = f(f(x))$

Principes clés

- **Taches** : Description d'une action unitaire : installer un paquet logiciel, créer un fichier depuis un template, télécharger un fichier ou un dépôt git, mettre en place une règle de pare-feu...
- **Play** : application de tâches sur des machines de l'inventaire.
- **Playbook** : Une liste de plays.
- **Inventaire** : Liste des machines gérées par Ansible.
- **Facts** : Faits sur la machine / l'infrastructure
- **Roles** :

Tâches

- **Définition** : Une brique élémentaire d'automatisation.
- **Rôle** : Elle appelle un module spécifique préférablement avec un nom qualifié complet (comme `ansible.builtin.apt`) pour réaliser une action précise sur un hôte cible.
- **Utilisation** : Les tâches s'écrivent à l'intérieur des playbooks ou des rôles.

Tâches - Syntaxe

Nom de la tâche et texte qui s'affichera dans le terminal

- **name:** Install Apache on Debian-based systems only

ansible.builtin.apt:

Le nom qualifié du module à exécuter

name: apache2

state: present

Arguments pour le module. Ici le nom de ce qui doit être installé avec apt, et l'état souhaité

when: ansible_os_family == "Debian"

Condition pour l'exécution de la tâche

Tâches - syntaxe

```
- name: Describe what this specific task does
ansible.builtin.copy:
  src: /local/path/to/file.txt
  dest: /remote/path/to/file.txt
  mode: '0644'
```

Avec une boucle !

```
- name: Install multiple security tools
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - fail2ban
    - ufw
    - clamav
```

Exemples

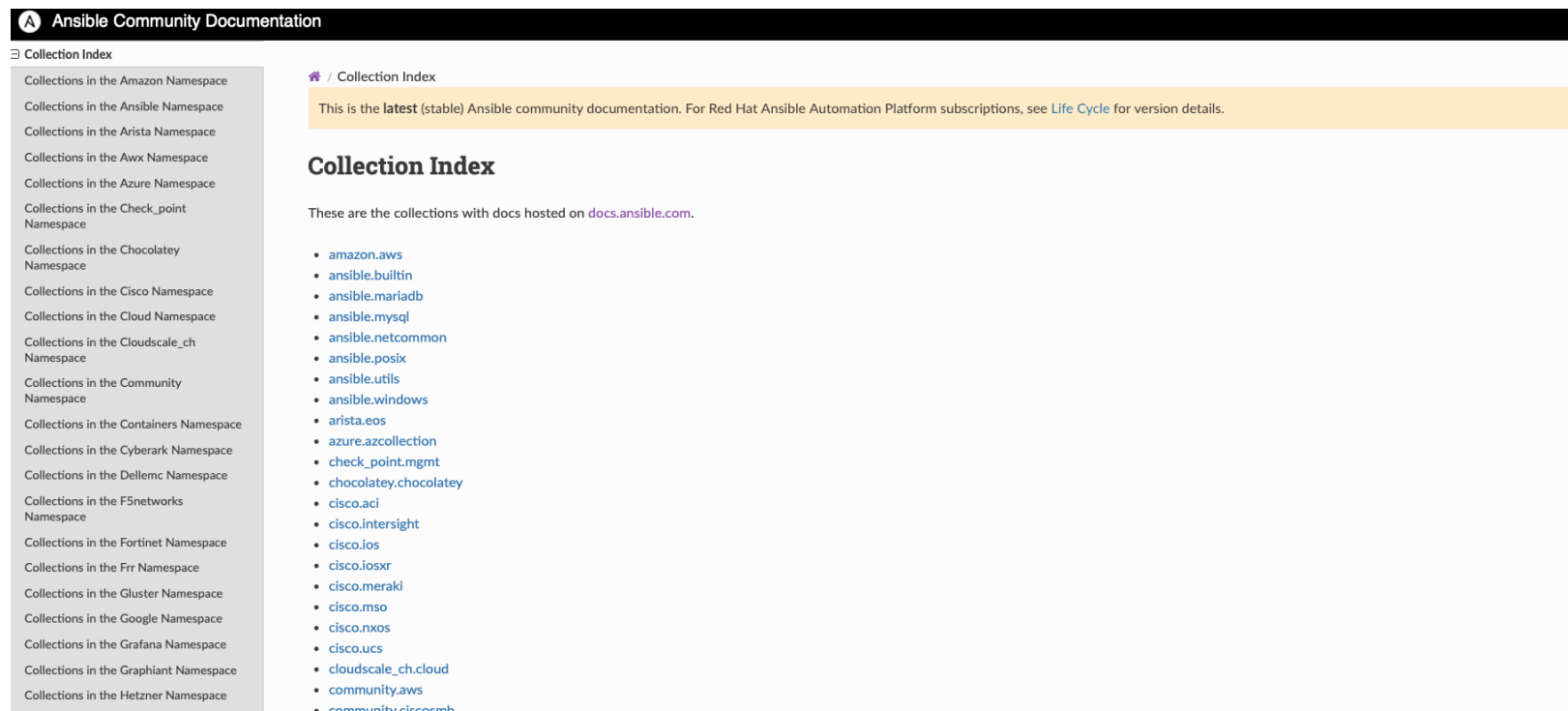
```
name: Make sure package ntp is not installed
ansible.builtin.apt:
  name: ntp
  state: absent
```

```
name: Make sure package python3 is up-to-date
ansible.builtin.apt:
  name: python3
  state: latest
```

```
name: Download file
ansible.builtin.get_url:
  url: https://go.dev/dl/go1.19.3.linux-amd64.tar.gz
  dest: /tmp/go1.19.3.linux-amd64.tar.gz
```

Tâches - Documentation

- <https://docs.ansible.com/projects/ansible/latest/collections/index.html>



The screenshot shows the 'Collection Index' page of the Ansible Community Documentation. On the left is a sidebar with a list of namespaces including Amazon, Ansible, Arista, Awx, Azure, Check_point, Chocolatey, Cisco, Cloud, Cloudscale_ch, Community, Containers, Cyberark, Dellemc, F5networks, Fortinet, Frr, Gluster, Google, Grafana, Graphiant, and Hetzner. The main content area has a header 'Collection Index' and a note that this is the latest (stable) documentation. Below this, it lists collections hosted on docs.ansible.com, including amazon.aws, ansible.builtin, ansible.mariadb, ansible.mysql, ansible.netcommon, ansible.posix, ansible.utils, ansible.windows, arista.eos, azure.azcollection, check_point.mgmt, chocolatey.chocolatey, cisco.aci, cisco.intersight, cisco.ios, cisco.iosxr, cisco.meraki, cisco.mso, cisco.nxos, cisco.ucs, cloudscale_ch.cloud, community.aws, and community.ciscosmb.

Ansible Community Documentation

Collection Index

Collections in the Amazon Namespace

Collections in the Ansible Namespace

Collections in the Arista Namespace

Collections in the Awx Namespace

Collections in the Azure Namespace

Collections in the Check_point Namespace

Collections in the Chocolatey Namespace

Collections in the Cisco Namespace

Collections in the Cloud Namespace

Collections in the Cloudscale_ch Namespace

Collections in the Community Namespace

Collections in the Containers Namespace

Collections in the Cyberark Namespace

Collections in the Dellemc Namespace

Collections in the F5networks Namespace

Collections in the Fortinet Namespace

Collections in the Frr Namespace

Collections in the Gluster Namespace

Collections in the Google Namespace

Collections in the Grafana Namespace

Collections in the Graphiant Namespace

Collections in the Hetzner Namespace

🌐 / Collection Index

This is the **latest** (stable) Ansible community documentation. For Red Hat Ansible Automation Platform subscriptions, see [Life Cycle](#) for version details.

Collection Index

These are the collections with docs hosted on docs.ansible.com.

- [amazon.aws](#)
- [ansible.builtin](#)
- [ansible.mariadb](#)
- [ansible.mysql](#)
- [ansible.netcommon](#)
- [ansible.posix](#)
- [ansible.utils](#)
- [ansible.windows](#)
- [arista.eos](#)
- [azure.azcollection](#)
- [check_point.mgmt](#)
- [chocolatey.chocolatey](#)
- [cisco.aci](#)
- [cisco.intersight](#)
- [cisco.ios](#)
- [cisco.iosxr](#)
- [cisco.meraki](#)
- [cisco.mso](#)
- [cisco.nxos](#)
- [cisco.ucs](#)
- [cloudscale_ch.cloud](#)
- [community.aws](#)
- [community.ciscosmb](#)

Inventaire

- On exécute un playbook sur un inventaire – i.e., un cluster de machines
- Dans un inventaire on peut déclarer des groupes de machines, de manière statique.
- Peut être déclaré sous plusieurs formats :
 - INI.
 - YAML.
- Possibilité d'ajouter des contraintes pour limiter l'application d'un playbook a un sous-ensemble d'un inventaire (groupe dans un inventaire)

Inventaire

```
[control]
control-node.lab ansible_host=10.10.20.10
```

Un groupe de machines

```
[webservers]
web1.lab ansible_host=10.10.20.21
web2.lab ansible_host=10.10.20.22
```

```
[dbservers]
db1.lab ansible_host=10.10.20.31
```

Déclaration du nom logique de la machine utilisable dans la commande et de l'IP pour la connexion SSH

```
[rhce_lab:children]
webservers
dbservers
```

Groupe parent qui agrège plusieurs sous groupes

Pour valider son inventaire:
`ansible-inventory -i inventory/hosts.yml --graph`

Limiter avec limit

Cibler tous les webservers

```
ansible-playbook -i inventory/hosts.yml site.yml --limit webservers
```

Cibler tous les webservers SAUF web1

```
ansible-playbook -i inventory/hosts.yml site.yml --limit  
'webservers:!web1.lab'
```

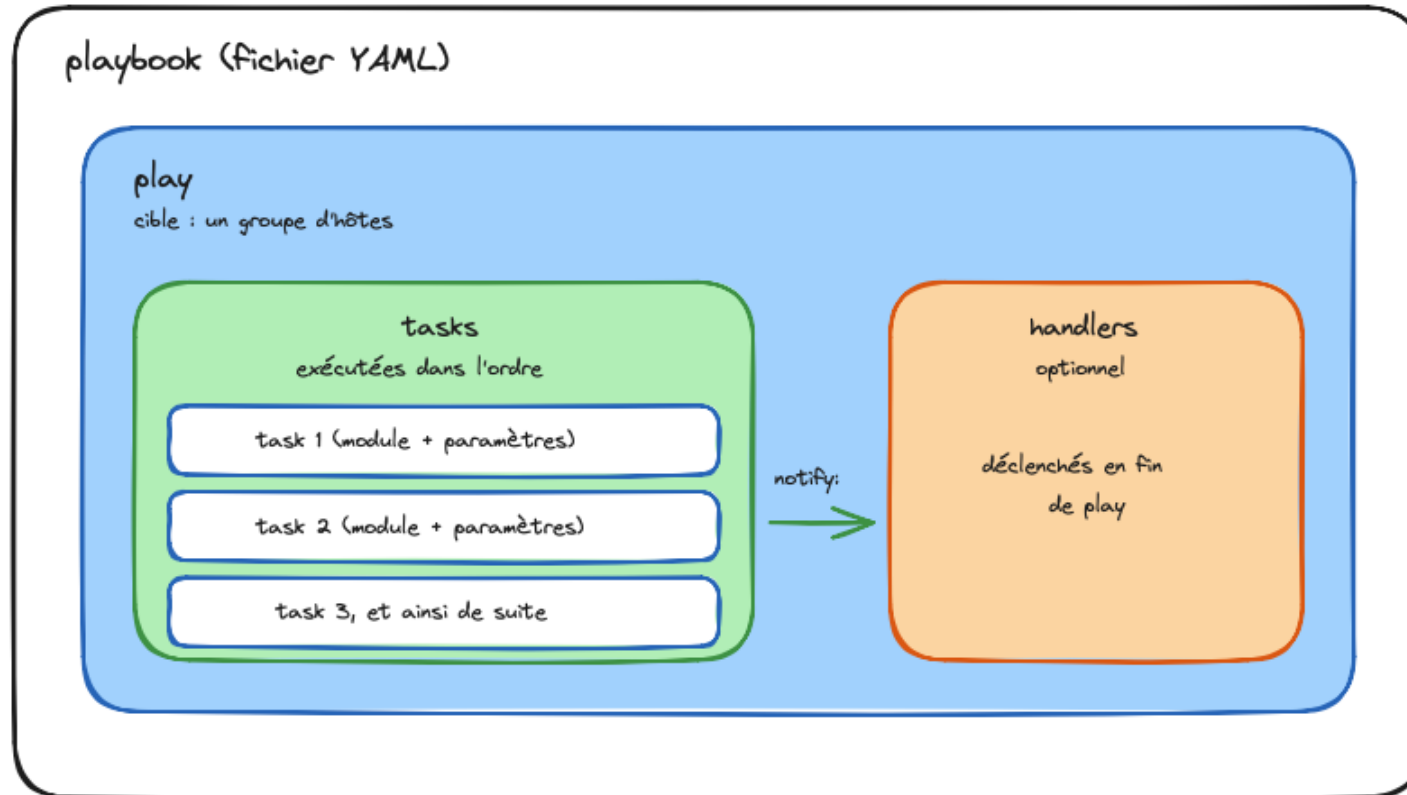
Intersection : hôtes qui sont à la fois dans webservers ET dans rhce_lab

```
ansible-playbook -i inventory/hosts.yml site.yml --limit  
'webservers:&rhce_lab'
```

Liste explicite

```
ansible-playbook -i inventory/hosts.yml site.yml --limit  
'web1.lab,db1.lab'
```

Playbook



Playbook

Clé	Rôle
<code>name:</code>	Description humaine du play (apparaît dans le PLAY RECAP)
<code>hosts:</code>	Pattern d'hôtes ciblé (<code>all</code> , <code>webserver</code> s , <code>web1.lab:!web2.lab</code> ...)
<code>become:</code>	Élévation <u>sudo</u> si <code>true</code>
<code>gather_facts:</code>	Collecte les facts au début (<code>true</code> par défaut)
<code>vars:</code>	Variables locales au play
<code>tasks:</code>	Liste ordonnée de tâches
<code>handlers:</code>	Tâches déclenchées en fin de play par <code>notify:</code>

Exemple de Playbook

```
---
- name: Déployer nginx sur les webservers
  hosts: webservers
  become: true
  gather_facts: true

  tasks:
    - name: Installer le paquet nginx
      ansible.builtin.dnf:
        name: nginx
        state: present

    - name: Démarrer et activer nginx au boot
      ansible.builtin.systemd:
        name: nginx
        state: started
        enabled: true

    - name: Ouvrir le port HTTP (80) dans firewalld
      ansible.posix.firewalld:
        service: http
        permanent: true
        immediate: true
        state: enabled

    - name: Vérifier que nginx répond sur localhost
      ansible.builtin.uri:
        url: http://localhost
        status_code: 200
        register: nginx_response
```

Le groupe sur lequel exécuter les tâches sur la base de l'inventaire

On exécute les tâches en sudo

Exécute le module setup au démarrage et expose les facts

Les tâches à exécuter séquentiellement

Executer un Playbook

Vérifier les changements

```
ansible-playbook --check demo.yml
```

Appliquer les changements

```
ansible-playbook demo.yml
```

Charger un inventaire particulier défini dans ansible.cfg

```
ansible-playbook -i inventory.yml demo.yml
```

Appliquer les changements à un sous-ensemble de machines

```
ansible-playbook -l vm1 demo.yml
```

Facts

- **peut collecter automatiquement des centaines de variables sur chaque managed node**
- **distribution, version, IP, mémoire, CPU, montages, packages, services, utilisateurs.**
- passer du playbook qui cible 1 hôte au playbook qui **s'adapte automatiquement** à la distribution, à la mémoire disponible, à la composition du groupe, sans aucune config manuelle.

Principaux facts

- Lister tous les facts disponibles :
`ansible all -i hosts.yml -u oki -m "setup"`

Fact	Type	Exemple
<code>ansible_distribution</code>	string	AlmaLinux , Ubuntu , Debian
<code>ansible_distribution_version</code>	string	10.1
<code>ansible_distribution_major_version</code>	string	10
<code>ansible_os_family</code>	string	RedHat , Debian
<code>ansible_default_ipv4.address</code>	string	10.10.20.21
<code>ansible_all_ipv4_addresses</code>	list	["10.10.20.21"]
<code>ansible_memtotal_mb</code>	int	1740
<code>ansible_processor_cores</code>	int	1
<code>ansible_processor_count</code>	int	1
<code>ansible_kernel</code>	string	5.14.0-...el10.x86_64
<code>ansible_hostname</code>	string	web1 (court)
<code>ansible_fqdn</code>	string	web1.lab (FQDN)
<code>ansible_python_version</code>	string	3.12.x
<code>ansible_pkg_mgr</code>	string	dnf , apt

Comment utiliser les facts

```
- name: Installer Apache sur RedHat
  ansible.builtin.yum:
    name: httpd
    state: present
  when: ansible_facts['os_family'] == "RedHat"
```

```
---
- name: "[IP v4 listing]"
  hosts: all
  gather_facts: yes
  tasks:
    - name: "[IP v4 listing] - generate html"
      template:
        src: listing_ipv4.html.j2
        dest: "/var/www/html/{{ inventory_hostname }}.html"
      connection: local
```

Handlers

- Un handler est une tâche qui s'exécute si une autre tâche l'a notifiée.

```
# ❌ Anti-pattern : restart à chaque run, même si rien n'a changé
- name: Mettre à jour nginx.conf
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf

- name: Redémarrer nginx
  ansible.builtin.systemd:
    name: nginx
    state: restarted # ← exécuté CHAQUE FOIS
```

```
# ✅ Pattern restart-on-config-change avec handler
tasks:
  - name: Mettre à jour nginx.conf
    ansible.builtin.template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: Recharger nginx # ← notifie le handler

handlers:
  - name: Recharger nginx
    ansible.builtin.systemd:
      name: nginx
      state: reloaded
```

tasks:

- name: Mettre à jour la config nginx
ansible.builtin.template:
src: nginx.conf.j2
dest: /etc/nginx/nginx.conf
notify: "config nginx changée"
- name: Mettre à jour le vhost
ansible.builtin.template:
src: site.conf.j2
dest: /etc/nginx/conf.d/site.conf
notify: "config nginx changée"

handlers:

- name: Recharger nginx
ansible.builtin.systemd:
name: nginx
state: reloaded
listen: "config nginx changée"
- name: Recharger fail2ban (qui dépend de nginx)
ansible.builtin.systemd:
name: fail2ban
state: reloaded
listen: "config nginx changée"

Rôles

- Pour faciliter la réutilisation (et rendre les playbooks plus lisibles).
- Plutôt que de regrouper toutes vos tâches, etc. dans un seul playbook, un rôle découpe la logique en une arborescence de dossiers.
- Convention de nommage et structure stricte
 - Squelette générable avec : `ansible-galaxy role init mon_serveur_web`

```
mon_role/  
├── defaults/  
│   └── main.yml    # Variables par défaut (faible priorité, facilement modifiables)  
├── vars/  
│   └── main.yml    # Variables internes au rôle (haute priorité)  
├── tasks/  
│   └── main.yml    # Liste des actions/tâches exécutées par le rôle  
├── handlers/  
│   └── main.yml    # Actions déclenchées par un 'notify' (ex. redémarrer un service)  
├── templates/  
│   └── app.j2      # Fichiers de configuration dynamiques (Jinja2)  
├── files/  
│   └── app.conf     # Fichiers statiques à copier tels quels  
└── meta/  
    └── main.yml     # Métadonnées du rôle et dépendances
```

Rôles

- Définitions de tâches pour un rôle `mon_service`.
 - Dans le fichier : `mon_service/tasks/main.yml`
 - Même principe pour les variables, etc.

- Utilisation du rôle :

```
- hosts: webservers
  become: true
  roles:
    - role: mon_serveur_web
      vars:
        port_ecoute: 8080
```

```
- name: Installer Nginx
  ansible.builtin.package:
    name: nginx
    state: present

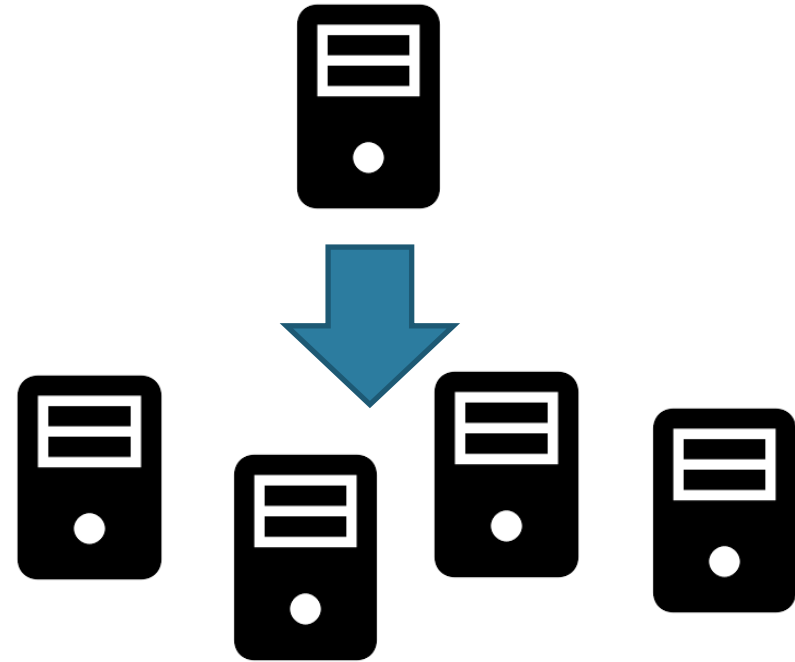
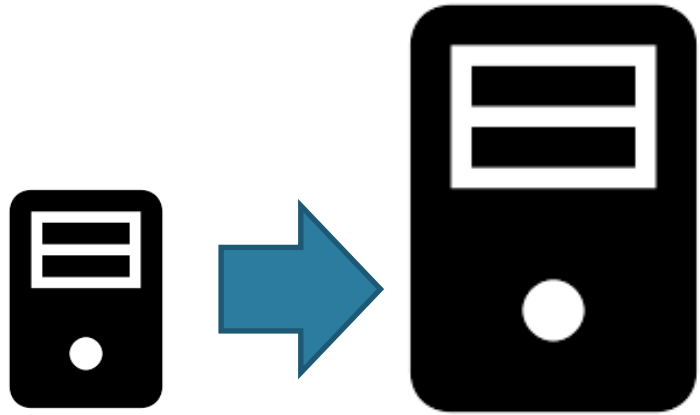
- name: Copier la configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Redemarrer Nginx
```

Maintenir son application

Passage à l'échelle & mise à jour.

Passage à l'échelle

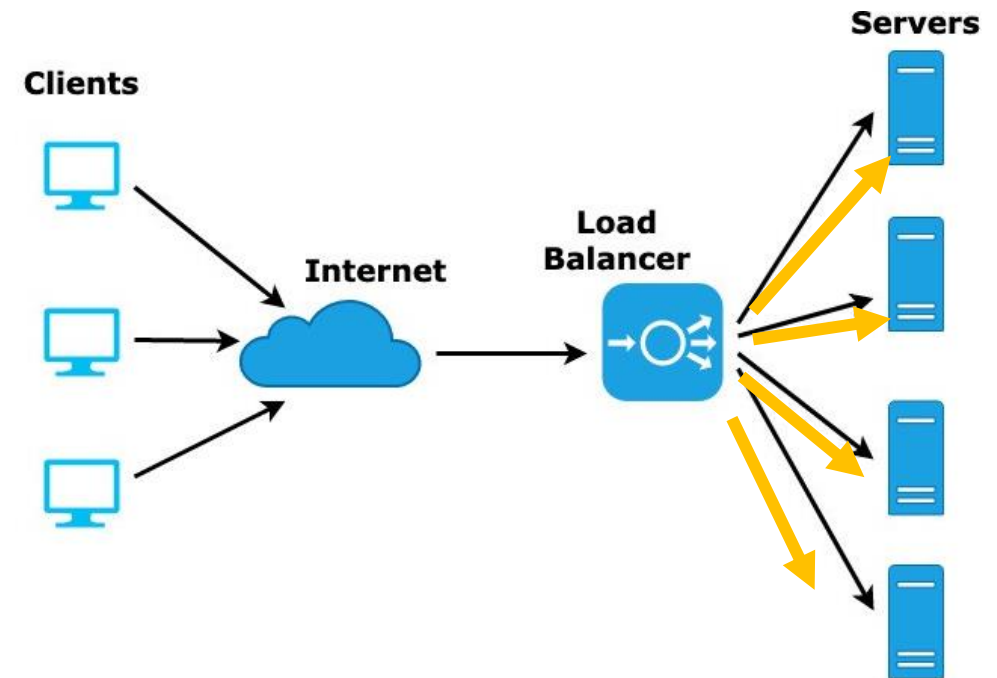
- Plusieurs techniques pour passer à l'échelle



Répliquations:

Passage à l'échelle horizontal

- Possible car application bien conçue
 - *e.g., on fait des services !*
- **Idée:** répliquer les services et distribuer la charge entre les instances du service.
 - **Répliquer les services :** sur le même (attention aux ports !) ou différents hôtes.
 - **Distribuer la charge :** load balancer redirige les requêtes. Applique une stratégie de distribution des requêtes. Ne redirige les requêtes que vers des instances **en bon état de fonctionnement**.



Répliquations:

Passage à l'échelle horizontal

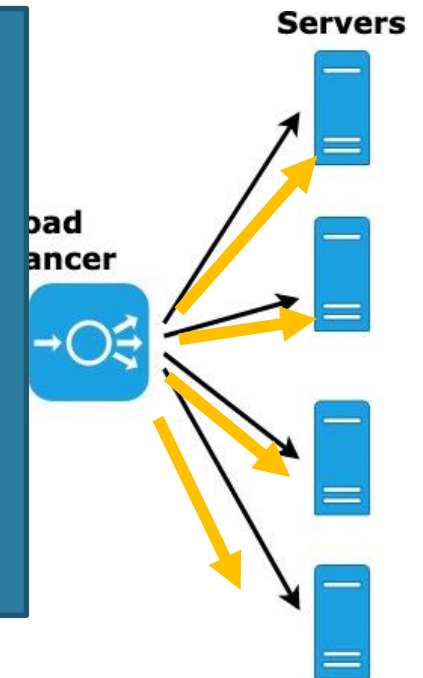
- Possible car application bien conçu
 - e.g., on fait des services /

- **Idée:** réplique charge entre

- **Répliquer les** ports!) ou dif
- **Distribuer la** requêtes. Ap requêtes. Ne instances en bon état de fonctionnement.

Quelques bénéfices :

- Disponibilité (health check + redirection).
- Performances (répartition charge).
- Adapter l'infrastructure à la charge.



Réplication et virtualisation

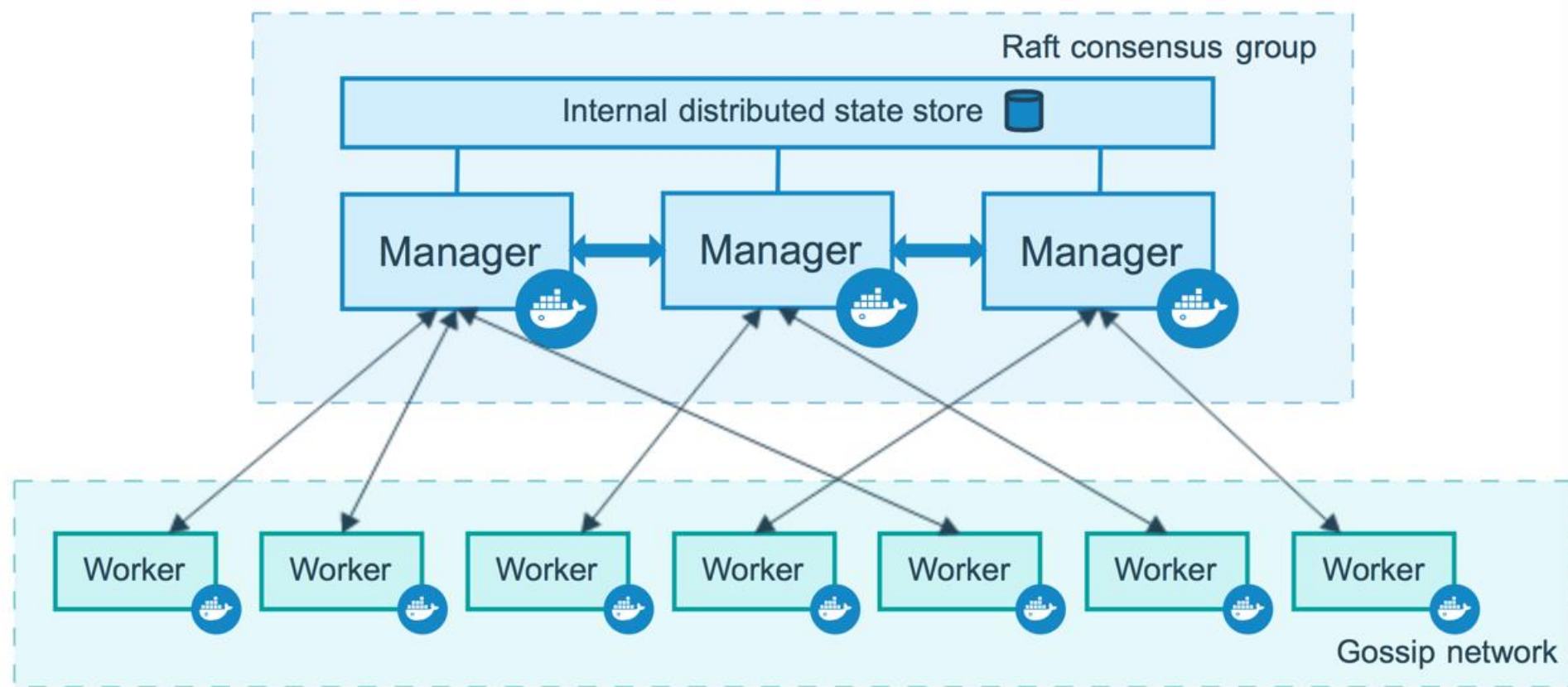
- La virtualisation facilite la mise en place du patron architectural vu précédemment.
- La virtualisation permet de:
 1. **Rapidement créer/supprimer de nouvelles instances** d'un service avec une configuration prédéfinie (rappel une image est immutable).
 2. Dans un **environnement isolé**.
 3. Permet de **rediriger les ports** (e.g., pour des services sur la même machine).
- En combinant le patron architectural, 1, 2, et 3, on peut mettre en place de nouvelles stratégies de déploiement.

Load Balancer

- De nombreuses stratégies existent, parmi les plus simples :
 - **Statiques** : sans prendre en compte l'état/la charge du système; e.g.:
 - *Round-robin* : cf. exemple précédent.
 - *Aléatoire* : on distribue les requêtes aléatoirement aux workers.
 - **Dynamiques** : en considérant l'état/la charge du système; e.g.:
 - *Master-worker* : un maître distribue la charge de travail à l'ensemble des travailleurs, en tenant compte de leur charge.

Docker Swarm

- Un mode de Docker pour gérer un cluster de (daemon) dockers.



Docker Swarm

- Permet de déployer une orchestration de containers (à la compose), tout en :
 - Configurant automatiquement un **multi-host overlay network** pour les composants de votre application
 - Les containers sont distribués dans l'essaim (on ne contrôle pas nécessairement comment et on ne veut pas nécessairement le savoir !).
 - Offrant de la **scalabilité horizontale dynamique et automatique**.
 - S'assure que le bon nombre de réplicas est toujours déployé même si un membre du cluster ou un container crash.
 - Assurant des **Communications sécurisées** avec TLS authentication and encryptions.
 - Permettant de **rolling updates**.
 - Mise à jour avec zéro? downtime.

Concepts

- Les **managers** : Ce sont les nœuds gestionnaires du cluster.
- Les **workers** : Ils exécutent les *tâches* confiées par les *managers*
- Swarm repose sur les même concept et la même syntaxe que docker-compose :
 - Notion de **service**.
 - Possibilité de déployer une orchestration (ici on parlera de stack).

Configurer son « essaim »

- Passer en mode swarm et créer un nouvel essaim

L'ip qui sera mise en avant pour rejoindre l'essaim

```
$ docker swarm init --advertise-addr 192.168.99.100
Swarm initialized: current node (dxn1zf6l61qsb1josjja83ngz) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join \
      --token SWMTKN-1-49nj1cmql0jkz5s954yi3oex3nedyz0fb0xx14ie39trti4wxv-8v xv8rssmk743ojnwacrr2e7c \
      192.168.99.100:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

- Rejoindre un essaim en tant que worker

```
$ docker swarm join \
  --token SWMTKN-1-49nj1cmql0jkz5s954yi3oex3nedyz0fb0xx14ie39trti4wxv-8v xv8rssmk743ojnwacrr2e7c \
  192.168.99.100:2377

This node joined a swarm as a worker.
```

Configurer son essaim

- Rejoindre un essaim en tant que manager :
 1. Récupérer le bon token depuis un manager

```
$ docker swarm join-token manager
```

To add a manager to this swarm, run the following command:

```
docker swarm join \
--token SWMTKN-1-61ztec5kyafptydic6jfc1i33t37flcl4nuipzcursor96k7kby-5vy9t8u35tuqm7vh67lrz9xp6 \
192.168.99.100:2377
```

2. Rejoindre

```
$ docker swarm join \
--token SWMTKN-1-61ztec5kyafptydic6jfc1i33t37flcl4nuipzcursor96k7kby-5vy9t8u35tuqm7vh67lrz9xp6 \
192.168.99.100:2377
```

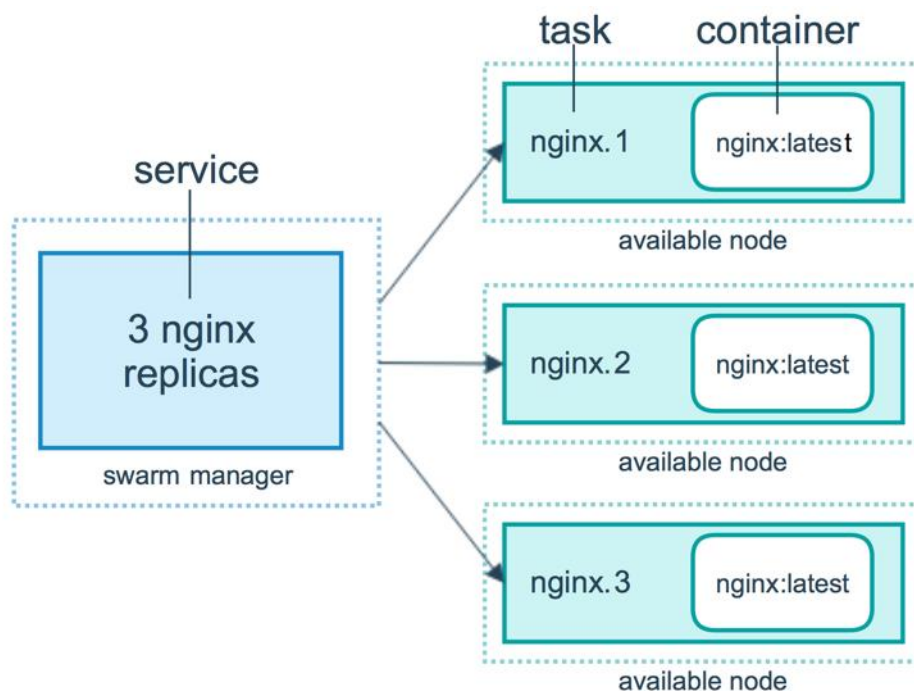
This node joined a swarm as a manager.

Indisponibilité

- Parfois, il est nécessaire de faire de la maintenance sur certains nœuds.
- Dans ces cas, on ne veut pas déployer de nouvelles tâches sur ces nœuds !
- On peut passer un nœud en mode DRAIN pour indiquer qu'il ne faut plus l'utiliser et déplacer les tâches sur d'autres nœuds :

```
docker node update --availability drain <NODE-ID>
```

Docker swarm et réplication



- Docker Swarm nativement offre un support pour :
 - La réplication.
 - Le rolling deployment.
- **Service** : **n containers** s'exécutant avec la même configuration et accessible via le même port.
- **Task** : un container + mécanisme de gestion du container (requêtes, healthcheck, etc.)

Créer et gérer un service

- Créer un service :

```
docker service create --name my_web --replicas 3 --publish 8080:80 nginx
```

- Logs d'un service :

```
docker service logs [OPTIONS] service|task
```

- Liste des services :

```
docker service ps [OPTIONS] service [service...]
```

- Répliquer un service :

```
docker service scale service=N [service=N...]
```

- Mettre à jour un service :

```
docker service update [OPTIONS] --image image[:tag] service
```

Déployer une stack (aka. une orchestration)

- Liste des nœuds dans le cluster :

```
docker node ls
```

- Déployer une stack :

```
docker stack deploy [OPTIONS] -c /path/to/file.yml stack
```

- Lister les services dans une stack :

```
docker stack services [OPTIONS] stack
```

- Supprimer une stack :

```
docker stack rm [OPTIONS] STACK [STACK...]
```

- Lister les stacks

```
docker stack ls
```

- Lister les tâches dans un stack

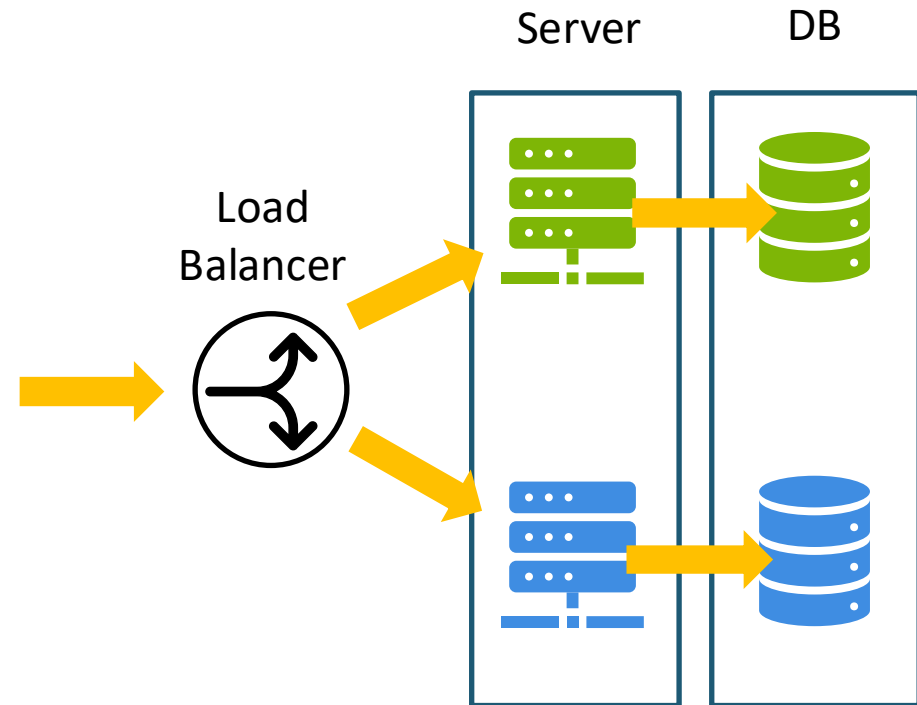
```
docker stack ps [OPTIONS] STACK
```

Stratégies de déploiement

- Maximiser la disponibilité du système en **minimisant l'impact** (downtime) **des mises à jours**.
- Pour approche de mise à jour plus **fiable** (réduire l'impact d'un problème lors de la mise à jour).
- 3 stratégies parmi les plus connues:
 1. Blue/green deployment
 2. Canary deployment
 3. Rolling deployment

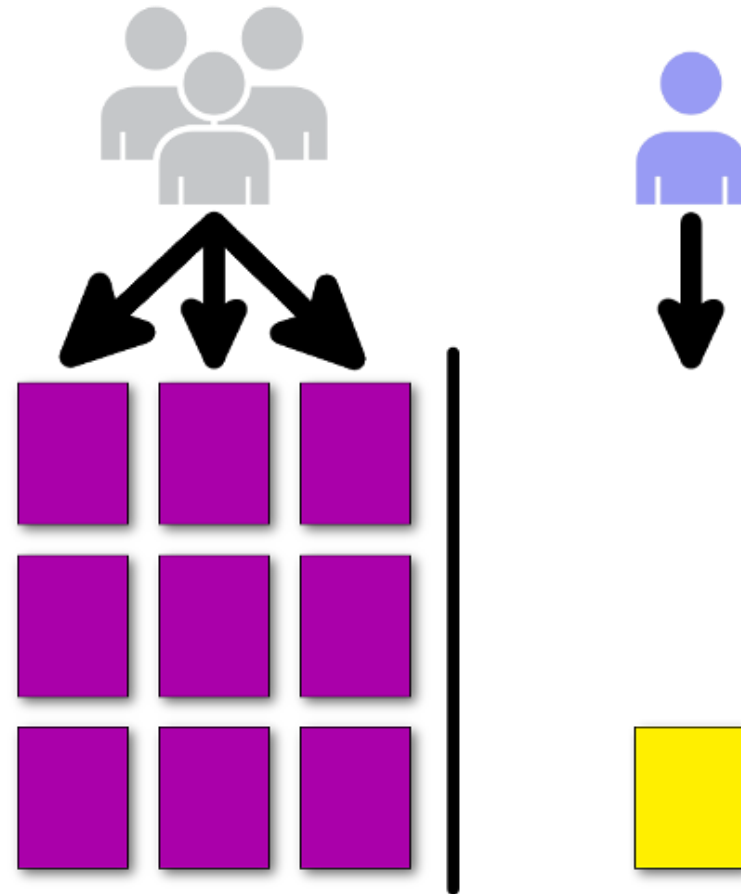
Blue/green deployment

- Créer deux environnements de production :
 - Le **bleu** avec l'**ancienne** version.
 - Le **vert** avec la **nouvelle** version.
- Utiliser un load balancer (répartisseur de charge) pour passer de l'un à l'autre :
 - **Zero-downtime**
 - **Rollback on failure**



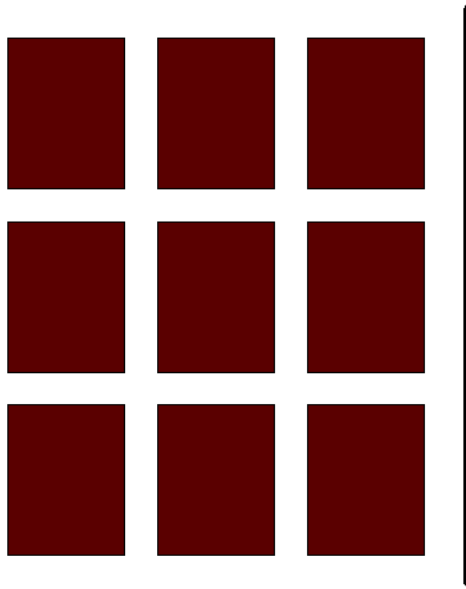
Canary deployment

- Fournir la nouvelle version d'un service **seulement à un sous ensemble des utilisateurs.**
- Observer les feedbacks et continuer si positifs.



Rolling update

- Proche du blue/green mais **on procède étape par étape.**
- On ne retire un service de la partie gauche **que lorsque validé** dans la partie droite.
- N+1 services.



Service update avec Docker Swarm

- Docker Swarm propose un mécanisme de rolling deployment

```
$ docker service update --image redis:3.0.7 redis  
redis
```

The scheduler applies rolling updates as follows by default:

- Stop the first task.
- Schedule update for the stopped task.
- Start the container for the updated task.
- If the update to a task returns **RUNNING**, wait for the specified delay period then start the next task.
- If, at any time during the update, a task returns **FAILED**, pause the update.

Maintenir son application

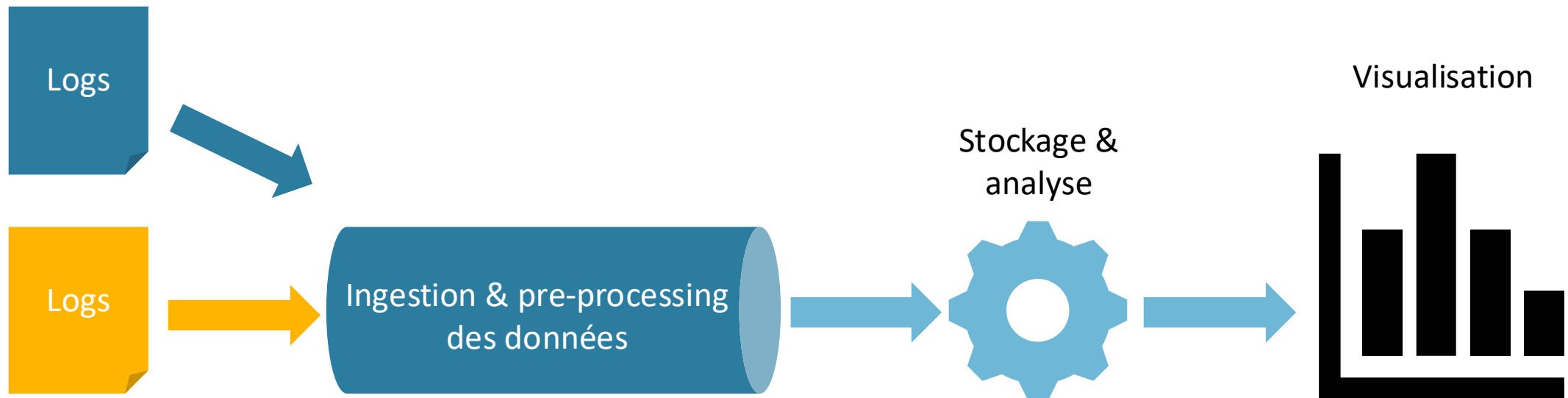
Monitoring.

Observer son application

- **Nécessaire pour maintenir son application :**
 - Connaitre le statut de votre système.
 - Identifier des problèmes de performances et savoir quand passer à l'échelle !
 - Identifier des bottleneck.
 - Observer la sécurité du système.
 - Identifier les points d'amélioration (architecture, performances, etc.)
- **Avant que les problèmes n'arrivent jusqu'aux utilisateurs finaux ...**
- **... ou réagir le plus rapidement possible.**

Pile de monitoring

- Typiquement constituée de trois grands éléments:



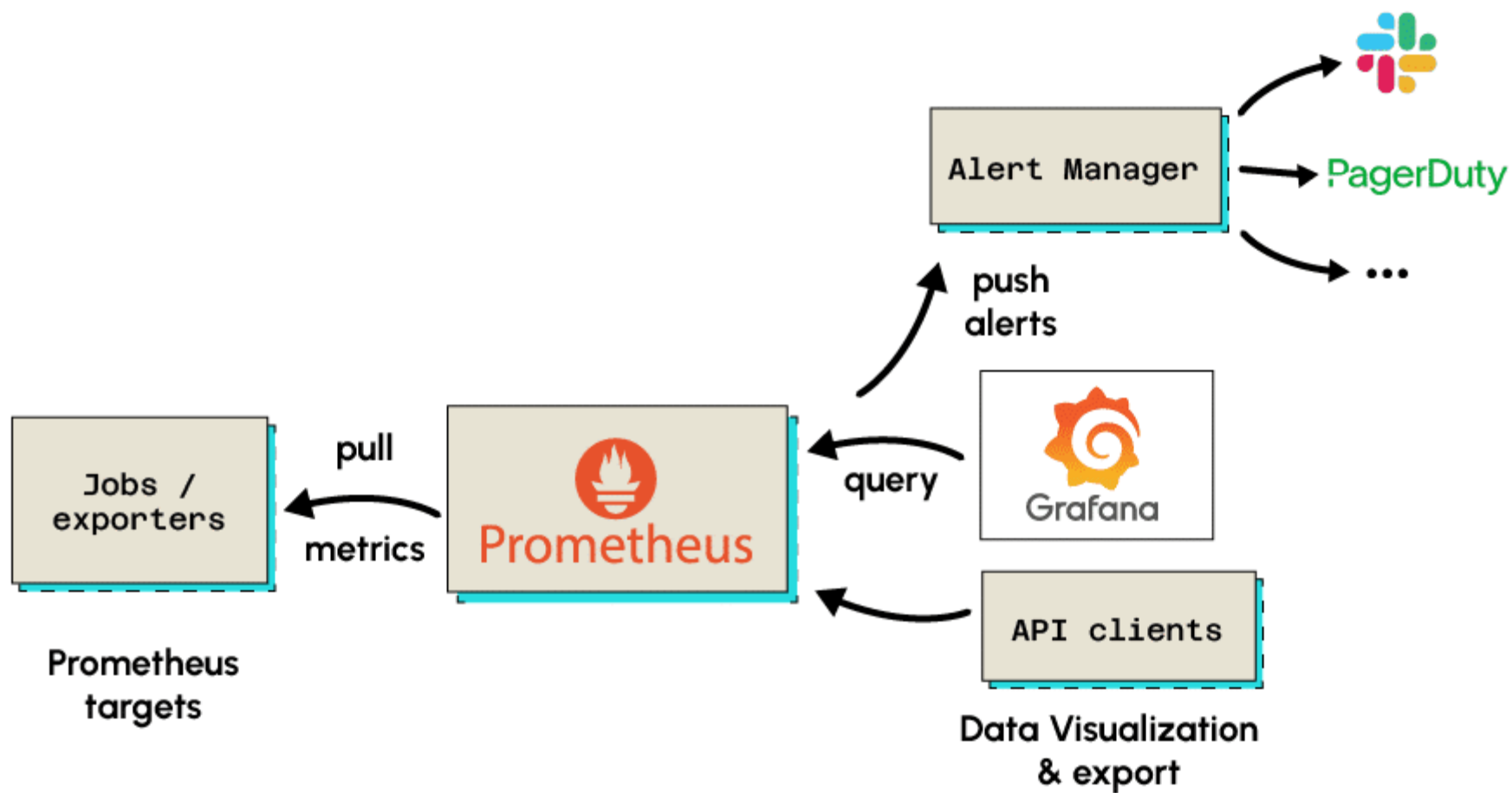
The ELK stack

- Une pile logicielle **gratuite** très populaire contenant (plus open source depuis 2021) :
 - **E = Elasticsearch**
 - Base de données et moteur de recherche de logs très performant basé sur Apache Lucene. Document-based datastore avec une grosse API REST
 - **L = Logstash**
 - Outil permettant ingérer et de transformer des données provenant de source diverses. Beaucoup de plugins d'analyse.
 - **K = Kibana**
 - Outil de visualisation.
1. Logstash ingère, transforme et envoie les données à la bonne destination.
 2. Elasticsearch indexe, analyse et recherche les données ingérées.
 3. Kibana visualise les résultats de l'analyse.

The Prometheus Stack

- Une autre pile open-source
- **Prometheus :**
 - collecte et stocke des données dans une base de données temporelle
- **Alert-manager :**
 - se connecte à Prometheus et permet l'émission d'alerts sur la base de règles de type ECA
- **Grafana :**
 - se connecte à Prometheus et l'utilise comme source de données pour la création de dashboard.

Prometheus



La suite/Les détails en TP !